

Barrett Viator  
DSE6211  
Professor Salemi  
March 9, 2025

## ***ABC Hotels Reservation Retention Risk: Final Report and Recommendations***

---

### ***Executive Summary***

ABC Hotels' Data Science team has conducted a machine learning case study of currently available client data to understand and predict customers who are more likely to not fulfill their reservation booking, leveraging demographic, financial, and behavioral metrics to identify at-risk clients. Key variables included lead time, segment type, arrival dates, number or special requests, duration of stay, and a customer's prior cancellation history. By comparing trends in fulfilled reservations versus canceled reservations, insights were derived via deep-learning neural networks to target customers at risk of canceling reservations with recommendations produced based on the machine learning models.

### ***Detailed Findings and Evaluation::***

The analysis utilized four deep-learning neural networks with varying parameters to address overfitting in order to produce more accurate results that can perform well with data that is procured about ABC Hotels' customer base. The final model selected utilized ReLU and Sigmoid activation functions with BCELoss as a loss function and RMSprop as the optimizer. Feature importance was derived from each model via the Captum library using integrated gradients to provide some direction for future models that do not include all of the features at once as a way to alleviate the unknown of the calculations derived in the neural network's hidden layers.

Three layers were ultimately selected at this stage for the model after encountering overfitting quickly with the initial attempts that included more layers for a deeper network. After testing multiple implementations of units in each layer, the best results were obtained with the most amount of comparable epochs reached by using 150 units in the primary layer and transitioning to 75 units for the output layer.

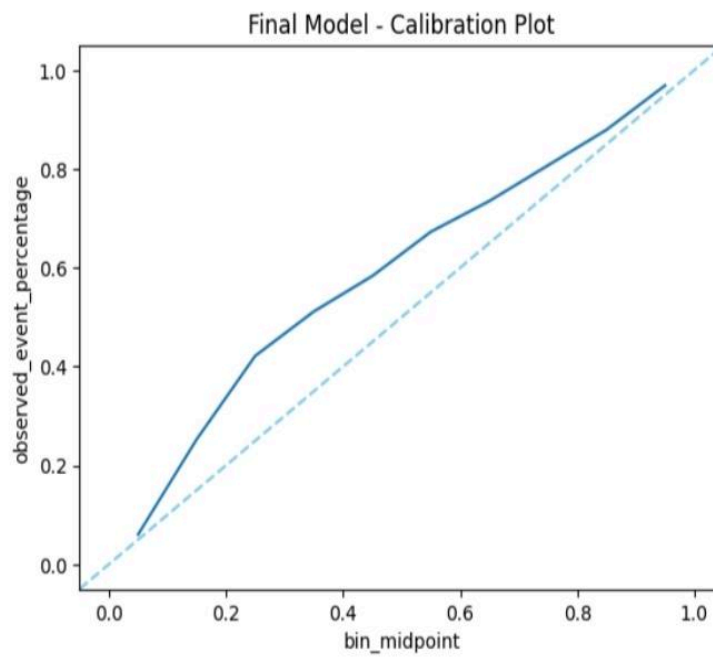
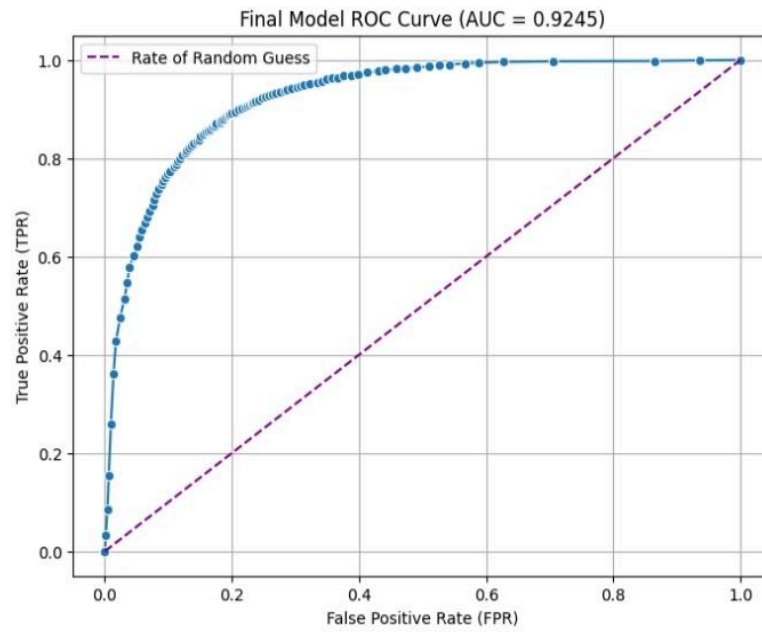
Further steps of implementing dropout, regularization, batch normalization, and a patience of 10 epochs were taken to resolve issues of overfitting seen in the other three models used for comparison in this evaluation. ReLU and Sigmoid activation functions were ultimately selected for this part of the binary classification problem due to their ability to work well with the loss

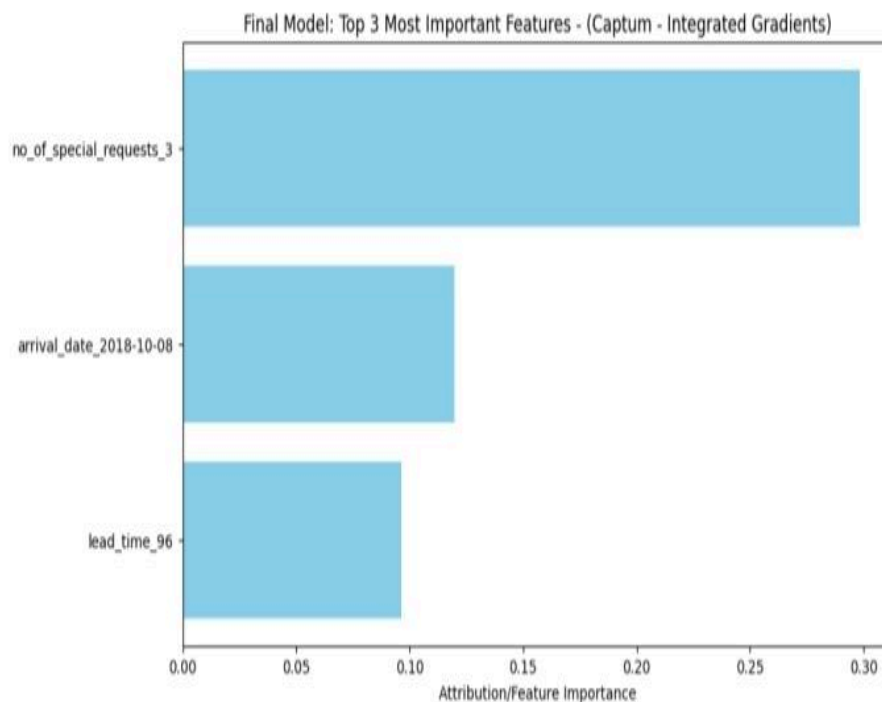
function method of BCEloss. RMSprop was selected for the optimization algorithm due to its ability to perform more robustly in comparison to SGD and Adam and the optimizers used to train the other three models in this study.

Early stopping concluded the training at 315 epochs with a training loss of 0.1183 and a validation loss of 0.3462. These outputs, compared to training and validation loss in the other three models indicates that overfitting has been resolved to the best of the Data Science team's abilities within the given time-frame. When comparing linearity and stability in the calibration plots, it was determined with certainty that this model was capable of outperforming the others. Results of the calibration plots can be seen below.

The final model had an accuracy of 85.99%, an ROC AUC Score of 92.45%, precision of 90.67% and recall of 84.40%. These results along with a more linear calibration plot can secure more certainty in the business decisions made based on the model when compared to the behaviors of the other models observed within the time constraints of this project.

### ***Final Model Figures***





### ***Recommendations:***

In regards to actions that can be taken from the insights gained from this machine learning analysis and future data, ABC Hotels should focus on building customer engagement and retention by developing tier-based loyalty programs for all customers to work towards qualifying for and direct special incentives to customers who are deemed most likely to cancel reservations to encourage continued engagement. Proactively enhancing customer engagement through regular, positive communication, not strictly limited to at-risk consumers, can strengthen relationships and foster trust. By leveraging insights from machine learning algorithms via deep-learning neural networks and further exploring these findings with predictive models in the future, ABC Hotels can implement targeted interventions to address specific customer needs and allocate resources more effectively, ensuring a greater impact on retention efforts and lessening the likelihood of an unfulfilled reservation. Further evaluation of new data every six to eight weeks would be ideal as a process of updating and maintaining a functional model that reflects the impacts of strategies implemented and to modify decisions as needed if trends in consumer behavior change.

Reducing reservations that are likely to be canceled provides opportunities for new customers to enjoy the experiences of ABC Hotels products and increases the overall reach of the organization. Implementing these recommendations and leveraging additional data through more finely-tuned deep-learning models and highly-accurate predictive modeling algorithms such as XG-boost and Random Forest can give ABC Hotels the ability to provide customers with a more

complete experience, strengthen relationships with those who are unsure of the benefits of our services, and improve outcomes for both stakeholders and clients.

---

### ***Detailed Reservation Retention Report***

Determining the factors that influence a customer to cancel reservations has grown more important for ensuring the longevity of ABC Hotels, and the ability to investigate the known behavioral factors of ABC Hotels' consumer base has evolved with the ability to leverage insights via machine learning and artificial intelligence. With the substantial data collected being available to this organization, insights can be uncovered to target specific customers who are deemed high risk for reservation cancellation to enhance positive outcomes for ABC Hotels' profitability and scalability. ABC Hotels now has the opportunity to take advantage of the latest technologies to procure more sound strategies that were not available prior to these advancements. Without taking these steps as an effort to improve the organization's standings financially and reputation wise among demographics that have yet to be reached, growth and retention may unnecessarily be stunted. By taking action based on the available insights returned by machine learning algorithms along with applying these methods to newly acquired data, the profitability and stability of the company can be promoted while ultimately providing a better customer experience by diverting efforts and resources more accurately towards consumers who would be more responsive to additional outreach and incentives.

### ***Approach & Data***

Developing an accurate feed forward neural network for deep learning requires the appropriate parameters, activation functions, loss functions, and optimization algorithms to be selected which is a task that involves comparing outputs of multiple models to solidify the certainty behind the predictions being made. Further addressing of model overfitting must be addressed as well through early stopping, drop out, and regularization because this ensures that the outcome derived by the model will perform better on future data with better precision.

Without the proper preparation of the data and the careful fine-tuning of the aforementioned aspects, business decisions can not be made in a truly data-driven method. The preliminary results of the deep learning model have provided the insights that it is possible to find trends within the data that support the efforts of targeting consumers with specific behaviors in order to prevent the amount of reservation cancellations that ABC Hotels is currently experiencing. The final model developed based on these findings improved upon the preliminary results and has been compared to three other models with varying parameters to strengthen the confidence in the findings.

The data collected about the consumers include valuable metrics that can provide great insights about future customers with similar demographics and preferences such as number of adults and number of children in the reservation, number of nights being stayed during the week as well as nights on the weekends, meal plan selection, required car parking spaces, and the type of room reserved. Lead time, arrival date, market segment type, repeat guest status, number of previous cancellations, number of previous bookings, average room price, number of special requests, and the current booking status can all be used as features with the machine learning model to find patterns of similarities that would not be possible for humans to calculate easily.

All of the aforementioned variables with the exception of “booking\_id” are included as features in the final supervised learning model to weigh their impact against the variable of “booking\_status” which measures active reservations versus cancelled reservations. Determining trends among cancelled reservations and then comparing them to factors of reservations that were successfully completed provides ABC Hotels with more accurate information needed to target high-risk consumers given similar behaviors of former consumers who cancelled their reservation.

Data quality determines the accuracy of machine learning outcomes when leveraging available information sources about our consumer base. Making business decisions based on historic predictors requires data to be modified into proper formats to accurately predict behaviors of future consumers actions. The validity of the data and the soundness of the machine learning methods enacted require data preprocessing for exploratory analysis through processes such as encoding, data splitting, scaling, and other data transformation methods to take the known information and to format it in a way that the algorithms can leverage valuable insights.

As with any project that requires analyzing data, removing duplicate data, resolving incomplete values, and coding qualitative variables must be accomplished as well. Outliers and high-leverage data points also need to be addressed when assessing if the data is being skewed by inputs that do not accurately represent the population of consumers of ABC Hotels. By properly cleaning and preprocessing the data, more accurate and valuable insights can be obtained. Future actions can be taken in the future by grouping variables to create new fields to provide actionable insights that were not initially available from the raw data.

### ***Detailed Findings and Evaluation:***

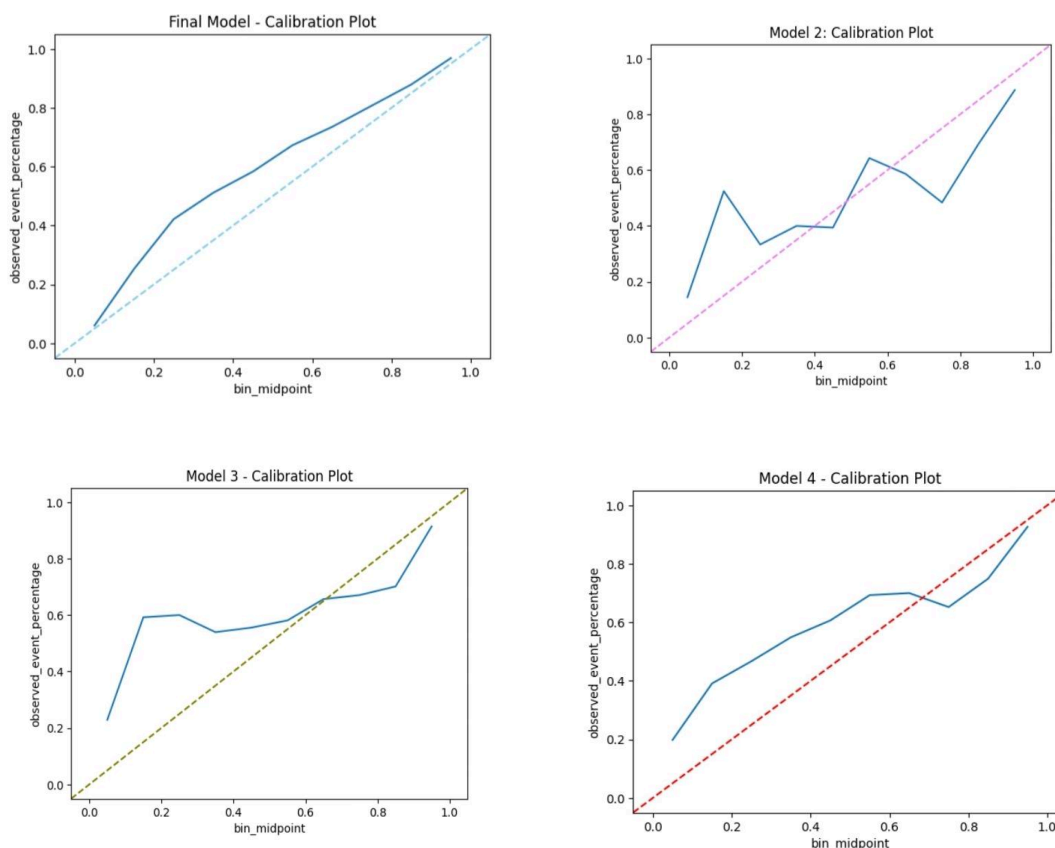
In order to produce an final model, three layers were ultimately selected at this stage for the model after encountering overfitting quickly with the initial attempts that included more layers for a deeper network. After testing multiple numbers of units in each layer, the best results were obtained with the most amount of comparable epochs reached by using 150 units in the primary layer and transitioning to 75 units for the output layer.

The model selected as the final version for the current steps took advantage of ReLU and Sigmoid activation functions with RMSprop as its optimizer and BCELoss as the loss function while implementing dropout, regularization, batch normalization, and a patience of 10 epochs to

resolve issues of overfitting seen in the other three models used for comparison in this evaluation. ReLU and Sigmoid activation functions were ultimately selected for this part of the binary classification problem due to their ability to work well with the loss function method of BCEloss. RMSprop was selected for the optimization algorithm due to its ability to perform more robustly in comparison to SGD and Adam and the optimizers used to train the other three models in this study.

Early stopping concluded the training at 315 epochs with a training loss of 0.1183 and a validation loss of 0.3462. These outputs, compared to training and validation loss in the other three models, indicates that overfitting has been resolved to the best of the Data Science team's abilities within the given timeframe. When comparing linearity and stability in the calibration plots, it was determined with certainty that this model was capable of outperforming the others. Results of the calibration plots can be seen below.

### *Model Calibration Plots*

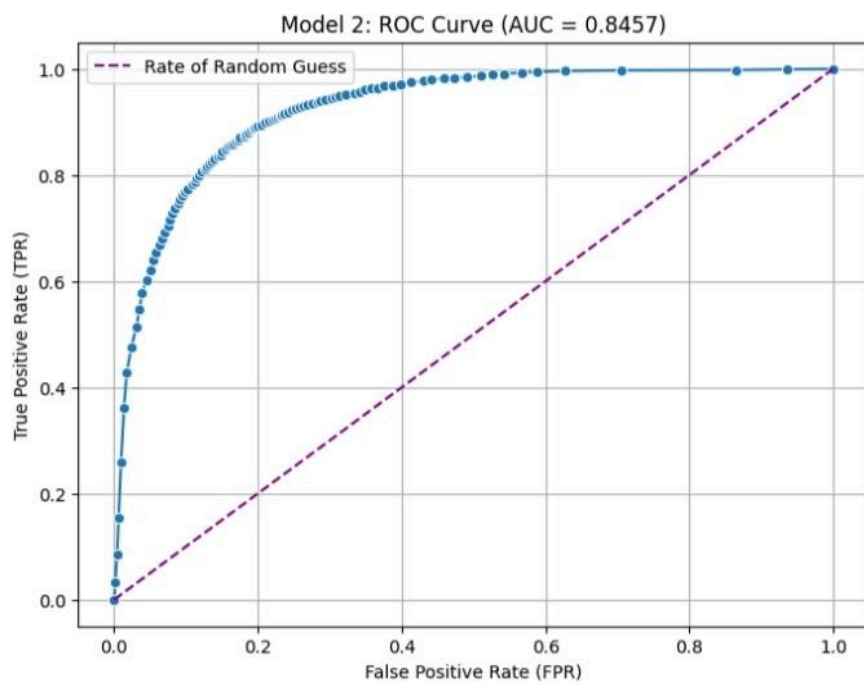
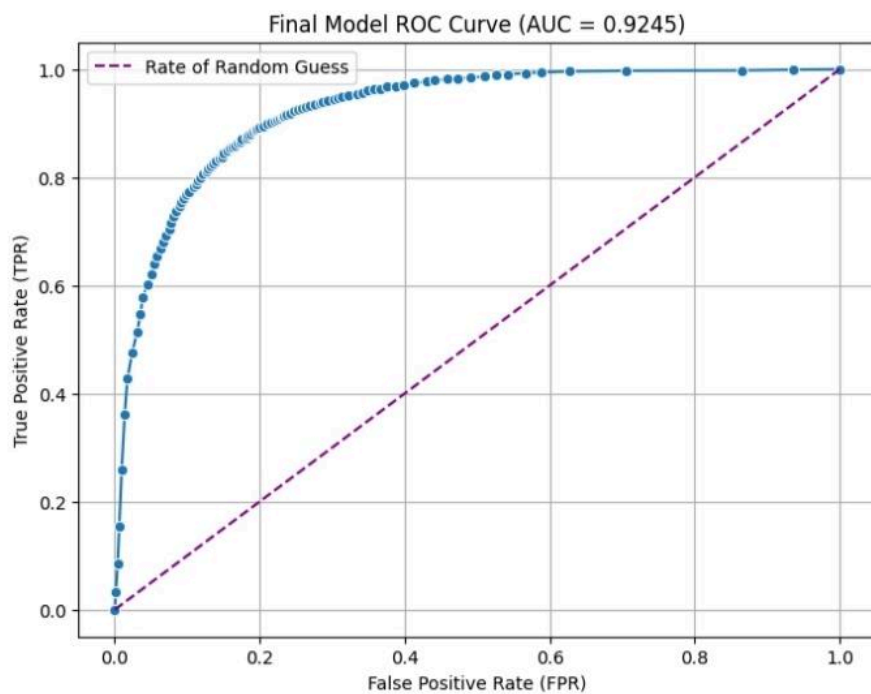


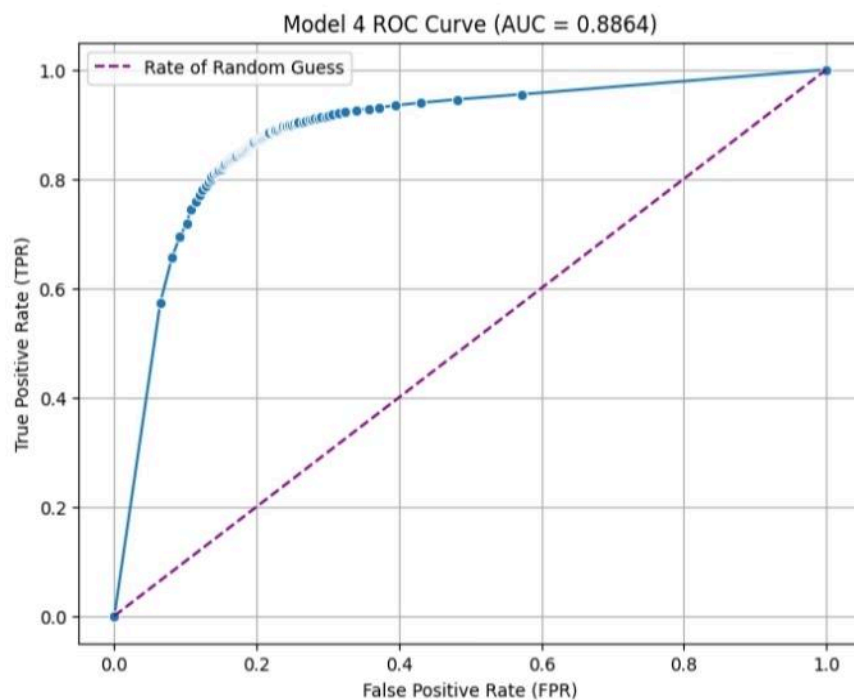
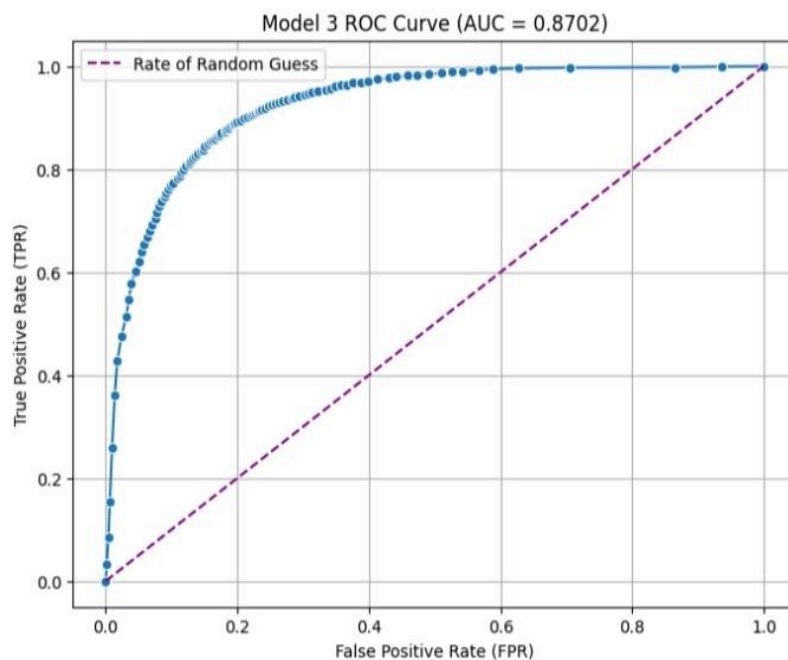
Model 2 uses ReLU and Sigmoid activation functions in three layers with 5000 epochs fully executed using BCELoss as the loss function. The training error signifies that the model is performing too accurately and the validation loss continues to increase every 100 epochs until

training is complete. Further testing was done in the third model consisting of a full 5000 epochs which introduced the need for a modified fourth model for evaluation that included early-stopping, dropout, and regularization as measures to improve performance even though the early-stopping was not triggered in the 200 epochs trained. Both Model 3 and Model 4 used Tanh and Sigmoid as activation functions and MSE loss as the loss function in their calculations. Model 4 used Adam as the optimizer while Model 3 used RMSProp. ROC AUC Scores, accuracy, precision, and recall can be seen for all models in the table and respective charts below.

### *Model Metrics*

|   | A           | B        | C       | D         | E      |
|---|-------------|----------|---------|-----------|--------|
| 1 | Model       | Accuracy | ROC AUC | Precision | Recall |
| 2 | Final Model | 85.99%   | 92.45%  | 90.67%    | 84.40% |
| 3 | Model 2     | 86.64%   | 84.57%  | 87.85%    | 93.15% |
| 4 | Model 3     | 84.11%   | 87.02%  | 89.69%    | 86.45% |
| 5 | Model 4     | 84.78%   | 86.64%  | 90.23%    | 86.92% |
| 6 |             |          |         |           |        |

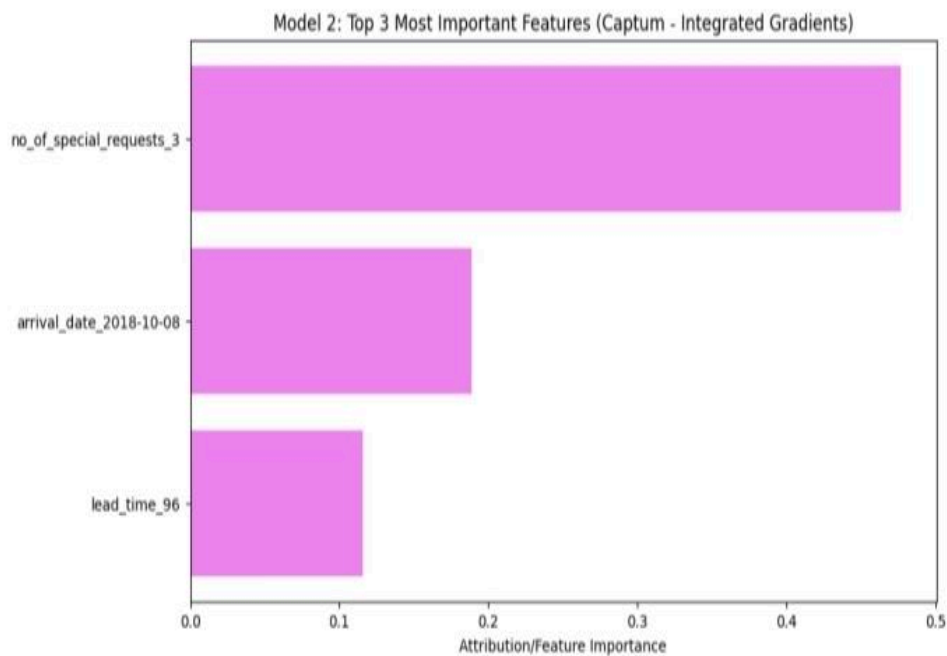
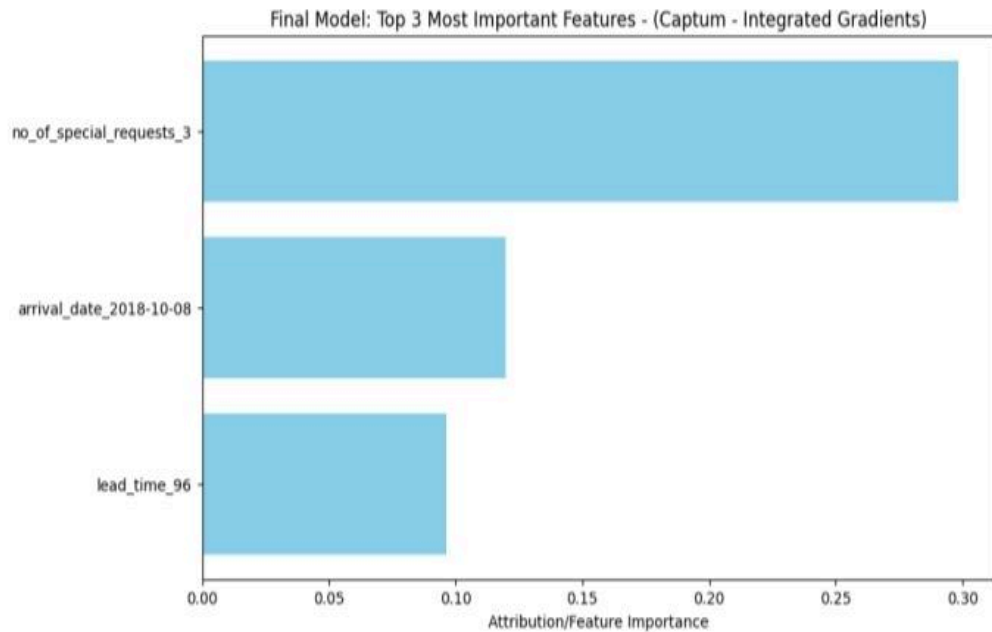
*Model ROC AUC Charts*

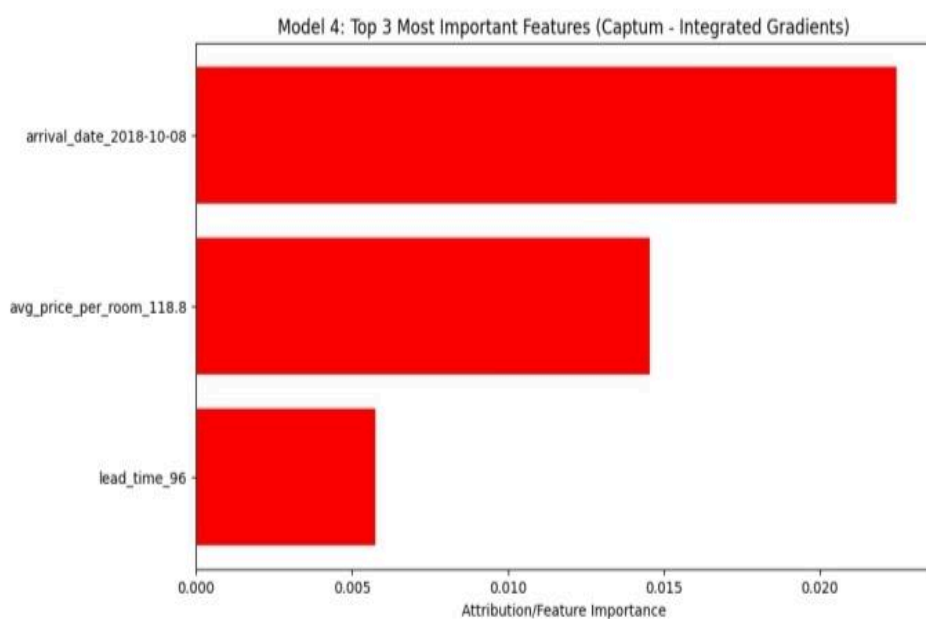
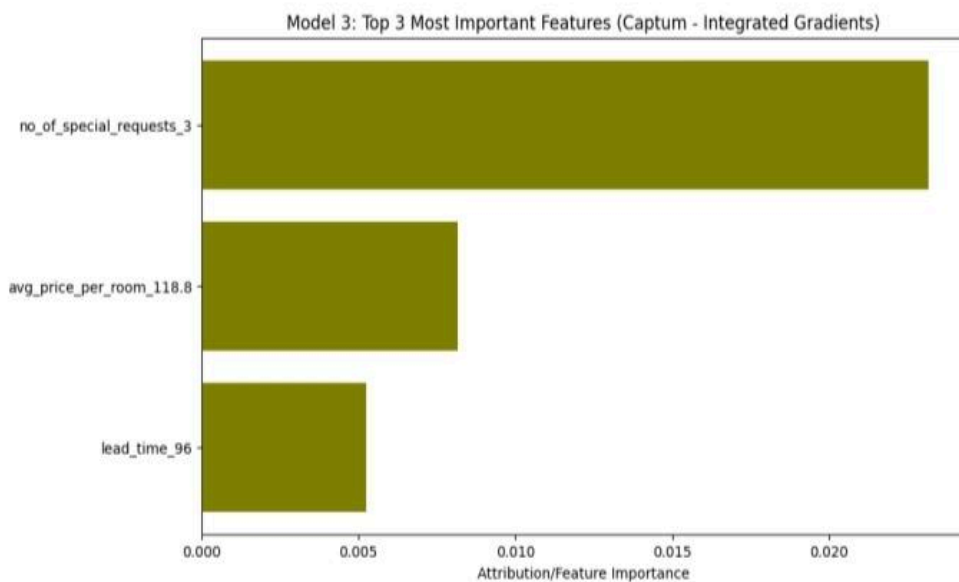


Using the Captum library to calculate Integrated Gradients to extract the top three feature importance in each model was leveraged to provide insights to which of the many features had the potential to produce the most impact on the models. All four models agreed that a

relationship existed with reservations with special requests being made and varied with importance on the date of the booking via either lead time or arrival date and cost of the room each night. Results from these steps in all four models can be evaluated via the charts below.

***Captum Integrated Gradients - Feature Importance***





Moving forward by developing deep learning neural network models with less features could create some clarity behind what is being done with the calculations in the hidden-layers of the neural networks. Confirming the findings of these future models by comparing it to outputs of predictive models such as XG-boost and Random Forest may also prove worthwhile when finding insights through new data that also consist of so many features needing to be explored

## ***Recommendations***

In a market where disposable income is becoming increasingly less available and where there are dozens of top-tier service providers for travel accommodations, it is imperative to develop a strategy that keeps consumers both excited about ABC Hotels and also motivated to follow through with their bookings. Building brand loyalty and increasing the amount of individuals referred to our services by friends, loved ones, and influencers online is one of the best strategies to secure consumer spending in this current economic environment. ABC Hotels can leverage the insights from the machine learning models to target customers who traditionally book reservations in advance, customers who tend to travel at specific times of the year, and customers who are known to make the effort to make special requests that fit their diverse needs. Alongside enhancing marketing efforts to these individuals, developing a rewards tier for repeat customers that is structured in a way that status elevation is achieved easily is also recommended to grow ABC Hotels' consumer base. Creative outreach to both high-risk consumers and loyalty program members is a strategy to maintain stakeholder engagement and excitement about our services.

Increased contact and outreach should not be the action taken only when the client is at risk of not fully experiencing the services and amenities ABC Hotels offers. These strategies should be part of actions taken when booking history is favorable and when there are opportunities to increase the favorable opinion of ABC Hotels in the eyes of these consumers who may not have had as positive of an experience as others even if it were out of the organization's control.

Increasing marketing about the benefits of the different tiers of the future loyalty program gives the consumers exciting goals to strive for and to think of ABC Hotels first when booking and reevaluating decisions during the travel planning process. These combined efforts can work synergistically to decrease the likelihood of consumers cancelling reservations that may have been able to be reserved by others who would not have decided to enjoy their experience elsewhere.

Consumers often have several options when it comes to deciding which hotel brand to rely on for their business and personal stays, and building positive relationships with responsive and timely customer service can also increase the odds for more favorable outcomes for both parties.

In order to prevent allocated resources from being used inefficiently, reevaluating the model every six to eight weeks is the ideal scenario to ensure that the variables used to target customers are the appropriate ones or to detect changes in consumer behavior that are different from the data this model trained on. Insights from new consumer patterns may become available as more data is collected and changes may appear as trends shift and markets fluctuate.

## ***Conclusion***

Strategically increasing communications and marketing efforts via incentives to customers who are deemed high-risk for reservation cancellation can only improve the outcomes for both the consumers and ABC Hotels. Basing these business and resource-based decisions by acting

upon insights that are backed by statistically sound machine learning models can ultimately provide more soundly made directives that require the use of valuable time and resources. By fitting the appropriate machine learning models to the data currently available to ABC Hotels and maintaining it regularly with incoming data, ABC Hotels can more precisely direct their initiatives to consumers who will be more responsive to the efforts and who will be more likely to respond in the future with more loyalty to the brand and to recommend ABC Hotels' services to their peers.

---

# Final Report Appendix

March 9, 2025

```
[ ]: Import necessary libraries
```

```
[2]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import precision_score, recall_score
from sklearn.metrics import auc
import torch
import torch.nn as nn
import torch.optim as optim
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
from captum.attr import IntegratedGradients
```

```
[ ]: Load, prepare, clean and encode data
```

```
[3]: data = pd.read_csv('C:\Program Files\Merrimack\DSE6211\Project\project_data.
    ↪ csv')
df = data.drop('Booking_ID', axis=1)
```

```
[4]: X = df.drop('booking_status', axis=1)
y = df['booking_status']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25,
    ↪ random_state=123)
```

```
[5]: encoder = OneHotEncoder(handle_unknown='ignore')
encoder.fit(X_train[['market_segment_type', 'lead_time', 'type_of_meal_plan',
    ↪ 'room_type_reserved', 'arrival_date',
    'no_of_adults', 'no_of_children', 'no_of_weekend_nights',
    ↪ 'no_of_week_nights',
    'required_car_parking_space', 'repeated_guest',
    ↪ 'no_of_previous_cancellations',
```

```

        'no_of_previous_bookings_not_canceled',
        ↪ 'avg_price_per_room', 'no_of_special_requests']])
encoded_df = pd.DataFrame(encoder.transform(X_train[['market_segment_type',
        ↪ 'lead_time', 'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
        ↪ 'no_of_adults', 'no_of_children', 'no_of_weekend_nights',
        ↪ 'no_of_week_nights',
        ↪ 'required_car_parking_space', 'repeated_guest',
        ↪ 'no_of_previous_cancellations',
        ↪ 'no_of_previous_bookings_not_canceled',
        ↪ 'avg_price_per_room', 'no_of_special_requests']]).toarray(),
        columns=encoder.get_feature_names_out(['market_segment_type', 'lead_time',
        ↪ 'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
        ↪ 'no_of_adults', 'no_of_children', 'no_of_weekend_nights',
        ↪ 'no_of_week_nights',
        ↪ 'required_car_parking_space', 'repeated_guest',
        ↪ 'no_of_previous_cancellations',
        ↪ 'no_of_previous_bookings_not_canceled',
        ↪ 'avg_price_per_room', 'no_of_special_requests'])))
X_train = pd.concat([X_train.drop(['market_segment_type', 'lead_time',
        ↪ 'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
        ↪ 'no_of_adults', 'no_of_children', 'no_of_weekend_nights',
        ↪ 'no_of_week_nights',
        ↪ 'required_car_parking_space', 'repeated_guest',
        ↪ 'no_of_previous_cancellations',
        ↪ 'no_of_previous_bookings_not_canceled',
        ↪ 'avg_price_per_room', 'no_of_special_requests'], axis=1).
        ↪ reset_index(drop=True), encoded_df], axis=1)
X_train.columns

```

```

[5]: Index(['market_segment_type_aviation', 'market_segment_type_complementary',
        ↪ 'market_segment_type_corporate', 'market_segment_type_offline',
        ↪ 'market_segment_type_online', 'lead_time_0', 'lead_time_1',
        ↪ 'lead_time_2', 'lead_time_3', 'lead_time_4',
        ↪ ...,
        ↪ 'avg_price_per_room_349.63', 'avg_price_per_room_365.0',
        ↪ 'avg_price_per_room_375.5', 'avg_price_per_room_540.0',
        ↪ 'no_of_special_requests_0', 'no_of_special_requests_1',
        ↪ 'no_of_special_requests_2', 'no_of_special_requests_3',
        ↪ 'no_of_special_requests_4', 'no_of_special_requests_5'],
        ↪ dtype='object', length=4373)

```

```

[ ]: Scale and encode data and convert to tensors

```

```

[6]: scaler = StandardScaler()
        scaler.fit(X_train)

```

```
X_train_scaled = scaler.transform(X_train)
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns)
```

```
[7]: X_train_tensor = torch.tensor(X_train_scaled.values, dtype=torch.float32)
X_train_tensor
```

```
[7]: tensor([[ -0.0567, -0.1056, -0.2436, ..., -0.1382, -0.0470, -0.0149],
          [ -0.0567, -0.1056, -0.2436, ..., -0.1382, -0.0470, -0.0149],
          [ -0.0567, -0.1056, -0.2436, ..., -0.1382, -0.0470, -0.0149],
          ...,
          [ -0.0567, -0.1056, -0.2436, ..., -0.1382, -0.0470, -0.0149],
          [ -0.0567, -0.1056, -0.2436, ..., -0.1382, -0.0470, -0.0149],
          [ -0.0567, -0.1056,  4.1057, ..., -0.1382, -0.0470, -0.0149]])
```

```
[8]: encoded_df = pd.DataFrame(encoder.transform(X_test[['market_segment_type',
↳ 'lead_time', 'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
↳ 'no_of_adults', 'no_of_children', 'no_of_weekend_nights',
↳ 'no_of_week_nights',
↳ 'required_car_parking_space', 'repeated_guest',
↳ 'no_of_previous_cancellations',
↳ 'no_of_previous_bookings_not_canceled',
↳ 'avg_price_per_room', 'no_of_special_requests']])).toarray(),
columns=encoder.get_feature_names_out(['market_segment_type', 'lead_time',
↳ 'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
↳ 'no_of_adults', 'no_of_children', 'no_of_weekend_nights',
↳ 'no_of_week_nights',
↳ 'required_car_parking_space', 'repeated_guest',
↳ 'no_of_previous_cancellations',
↳ 'no_of_previous_bookings_not_canceled',
↳ 'avg_price_per_room', 'no_of_special_requests']))
X_test = pd.concat([X_test.drop(['market_segment_type', 'lead_time',
↳ 'type_of_meal_plan', 'room_type_reserved', 'arrival_date',
↳ 'no_of_adults', 'no_of_children', 'no_of_weekend_nights',
↳ 'no_of_week_nights',
↳ 'required_car_parking_space', 'repeated_guest',
↳ 'no_of_previous_cancellations',
↳ 'no_of_previous_bookings_not_canceled',
↳ 'avg_price_per_room', 'no_of_special_requests'], axis=1).
↳ reset_index(drop=True), encoded_df], axis=1)
X_test.columns
```

```
[8]: Index(['market_segment_type_aviation', 'market_segment_type_complementary',
          'market_segment_type_corporate', 'market_segment_type_offline',
          'market_segment_type_online', 'lead_time_0', 'lead_time_1',
          'lead_time_2', 'lead_time_3', 'lead_time_4',
```

```
...
'avg_price_per_room_349.63', 'avg_price_per_room_365.0',
'avg_price_per_room_375.5', 'avg_price_per_room_540.0',
'no_of_special_requests_0', 'no_of_special_requests_1',
'no_of_special_requests_2', 'no_of_special_requests_3',
'no_of_special_requests_4', 'no_of_special_requests_5'],
dtype='object', length=4373)
```

```
[9]: X_test_scaled = scaler.transform(X_test)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_train.columns)

X_test_tensor = torch.tensor(X_test_scaled.values, dtype=torch.float32)
```

```
[10]: encoder = LabelEncoder()
y_train_encoded = encoder.fit_transform(y_train)
y_test_encoded = encoder.transform(y_test)

y_train_tensor = torch.tensor(y_train_encoded, dtype=torch.float)
y_test_tensor = torch.tensor(y_test_encoded, dtype=torch.float)
```

```
[11]: y_train_tensor
y_test_tensor
```

```
[11]: tensor([1., 1., 1., ..., 1., 0., 1.])
```

```
[ ]: Shuffle data and prepare training and test sets for evaluation
```

```
[12]: n_samples = X_train_tensor.shape[0]
n_val = int(0.2 * n_samples)
shuffled_indices = torch.randperm(n_samples)
train_indices = shuffled_indices[:-n_val]
val_indices = shuffled_indices[-n_val:]
train_indices, val_indices

X_tr = X_train_tensor[train_indices]
y_tr = y_train_tensor[train_indices]
X_val = X_train_tensor[val_indices]
y_val = y_train_tensor[val_indices]
```

```
[ ]: Final Model using Batch Normalization, Dropout, Regularization, and Patience
ReLU and Sigmoid activation function, BCELoss loss function, and RMSProp as the
optimizer with stopping on 2000 epochs
```

```
[13]: seq_model = nn.Sequential(
    nn.Linear(4373, 150),
    nn.BatchNorm1d(150),
    nn.ReLU(),
```

```

nn.Dropout(0.5),

nn.Linear(150, 75),
nn.BatchNorm1d(75),
nn.ReLU(),
nn.Dropout(0.5),

nn.Linear(75, 1),
nn.Sigmoid()
)

```

```

[14]: def training_loop(n_epochs, optimizer, model, loss_fn, X_tr, X_val, y_tr,
    ↪y_val, patience=10):
    best_val_loss = float("inf")
    counter = 0

    for epoch in range(1, n_epochs + 1):
        seq_model.train()

        X_tr_p = model(X_tr)
        loss_train = loss_fn(X_tr_p, y_tr.unsqueeze(1))

        optimizer.zero_grad()
        loss_train.backward()
        optimizer.step()

        seq_model.eval()
        with torch.no_grad():
            X_val_p = model(X_val)
            loss_val = loss_fn(X_val_p, y_val.unsqueeze(1))

        if epoch == 1 or epoch % 10 == 0:
            print(f"Epoch {epoch}, Training loss {loss_train.item():.4f},
    ↪Validation loss {loss_val.item():.4f}")

            scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,
    ↪mode='min', patience=10, factor=0.5)

        if loss_val.item() < best_val_loss:
            best_val_loss = loss_val.item()
            counter = 0
        else:
            counter += 1

        if counter >= patience:

```

```

        print(f"Overfitting detected at epoch {epoch}. Training concluded.")
        print(f"Final Epoch: {epoch}, Training Loss: {loss_train.item():.4f}, Validation Loss: {loss_val.item():.4f}")
        break

```

```

[15]: optimizer = torch.optim.RMSprop(seq_model.parameters(), lr=1e-4,
    ↪weight_decay=1e-4)
training_loop(
    n_epochs = 2000,
    optimizer = optimizer,
    model = seq_model,
    loss_fn = nn.BCELoss(),
    X_tr = X_tr,
    X_val = X_val,
    y_tr = y_tr,
    y_val = y_val)

```

```

Epoch 1, Training loss 0.6840, Validation loss 0.6838
Epoch 10, Training loss 0.5166, Validation loss 0.5965
Epoch 20, Training loss 0.4328, Validation loss 0.5146
Epoch 30, Training loss 0.3866, Validation loss 0.4748
Epoch 40, Training loss 0.3549, Validation loss 0.4549
Epoch 50, Training loss 0.3299, Validation loss 0.4421
Epoch 60, Training loss 0.3122, Validation loss 0.4326
Epoch 70, Training loss 0.2929, Validation loss 0.4238
Epoch 80, Training loss 0.2782, Validation loss 0.4160
Epoch 90, Training loss 0.2646, Validation loss 0.4090
Epoch 100, Training loss 0.2515, Validation loss 0.4023
Epoch 110, Training loss 0.2408, Validation loss 0.3954
Epoch 120, Training loss 0.2303, Validation loss 0.3895
Epoch 130, Training loss 0.2205, Validation loss 0.3841
Epoch 140, Training loss 0.2107, Validation loss 0.3791
Epoch 150, Training loss 0.2035, Validation loss 0.3740
Epoch 160, Training loss 0.1954, Validation loss 0.3699
Epoch 170, Training loss 0.1869, Validation loss 0.3661
Epoch 180, Training loss 0.1795, Validation loss 0.3630
Epoch 190, Training loss 0.1749, Validation loss 0.3604
Epoch 200, Training loss 0.1683, Validation loss 0.3583
Epoch 210, Training loss 0.1624, Validation loss 0.3561
Epoch 220, Training loss 0.1581, Validation loss 0.3543
Epoch 230, Training loss 0.1511, Validation loss 0.3528
Epoch 240, Training loss 0.1469, Validation loss 0.3513
Epoch 250, Training loss 0.1428, Validation loss 0.3494
Epoch 260, Training loss 0.1377, Validation loss 0.3481
Epoch 270, Training loss 0.1337, Validation loss 0.3475
Epoch 280, Training loss 0.1287, Validation loss 0.3471
Epoch 290, Training loss 0.1257, Validation loss 0.3469
Epoch 300, Training loss 0.1233, Validation loss 0.3466

```

Epoch 310, Training loss 0.1193, Validation loss 0.3459  
Overfitting detected at epoch 315. Training concluded.  
Final Epoch: 315, Training Loss: 0.1183, Validation Loss: 0.3462

```
[16]: seq_model(X_test_tensor).detach().numpy()
```

```
[16]: array([[0.94555795],  
            [0.94871026],  
            [0.9244116 ],  
            ...,  
            [0.97000116],  
            [0.00560132],  
            [0.8507167 ]], dtype=float32)
```

```
[ ]: Final Model predictions, ROC AUC, Calibration Plot, and Feature Importance
```

```
[17]: p_1_actual = pd.DataFrame({'p_1': seq_model(X_test_tensor).detach().numpy().  
                                ↪reshape(-1), 'actual': y_test_encoded})  
p_1_actual.head()
```

```
[17]:
```

|   | p_1      | actual |
|---|----------|--------|
| 0 | 0.945558 | 1      |
| 1 | 0.948710 | 1      |
| 2 | 0.924412 | 1      |
| 3 | 0.348367 | 1      |
| 4 | 0.950845 | 1      |

```
[18]: roc_list = []  
  
actuals_0 = (p_1_actual['actual'] == 0).sum()  
actuals_1 = (p_1_actual['actual'] == 1).sum()  
  
for i in np.arange(1, -0.01, -0.01):  
  
    over_threshold = p_1_actual[p_1_actual['p_1'] >= i]  
  
    fp = (over_threshold['actual'] == 0).sum()  
    tp = (over_threshold['actual'] == 1).sum()  
  
    fpr = fp / actuals_0 if actuals_0 > 0 else 0.0  
    tpr = tp / actuals_1 if actuals_1 > 0 else 0.0  
  
    roc_list.append((i, fpr, tpr))  
  
roc_data = pd.DataFrame(roc_list, columns=['threshold', 'fpr', 'tpr'])  
  
roc_data
```

```
[18]:      threshold      fpr      tpr
0    1.000000e+00  0.000000  0.000000
1    9.900000e-01  0.001366  0.031969
2    9.800000e-01  0.004097  0.085957
3    9.700000e-01  0.006145  0.154461
4    9.600000e-01  0.009560  0.259827
..      ...      ...      ...
96    4.000000e-02  0.626494  0.995922
97    3.000000e-02  0.704677  0.997227
98    2.000000e-02  0.865483  0.997880
99    1.000000e-02  0.936156  0.999184
100 -8.881784e-16  1.000000  1.000000
```

[101 rows x 3 columns]

```
[19]: roc_auc = auc(roc_data['fpr'], roc_data['tpr'])
print(f"ROC AUC Score: {roc_auc:.4f}")
```

ROC AUC Score: 0.9245

```
[20]: predicted_labels = (p_1_actual['p_1'] >= 0.5).astype(int)

accuracy = (predicted_labels == p_1_actual['actual']).mean()

print(f"Accuracy: {accuracy:.4f}")
```

Accuracy: 0.8599

```
[21]: y_pred_probs = seq_model(X_test_tensor).detach().numpy()

y_pred = (y_pred_probs >= 0.5).astype(int)

y_true = y_test_tensor.numpy().astype(int)
```

```
[22]: from sklearn.metrics import precision_score, recall_score

precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)

print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
```

Precision: 0.9067

Recall: 0.8840

```
[81]: plt.figure(figsize=(8, 6))
roc = sns.lineplot(x='fpr', y='tpr', data=roc_data, marker='o', linestyle='-')

xlims = roc.get_xlim()
```

```

ylims = roc.get_ylim()

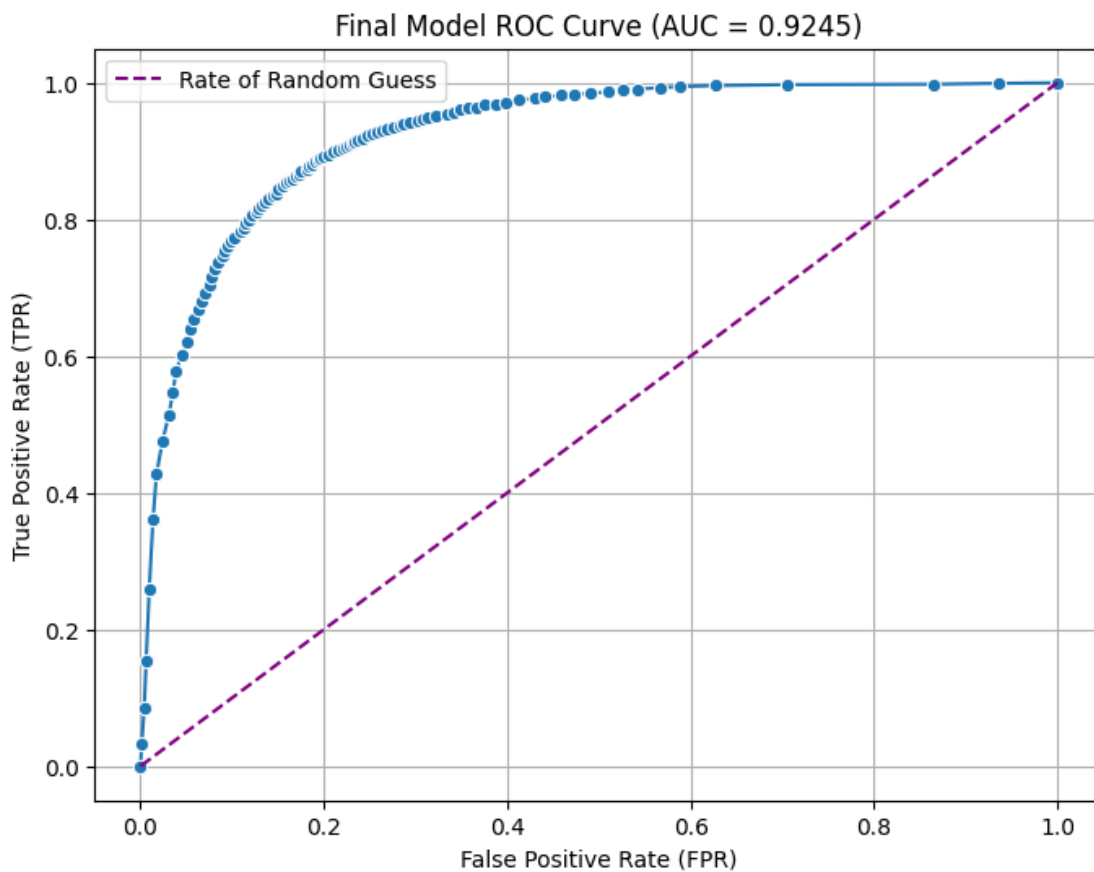
plt.xlim(-0.05, 1.05)
plt.ylim(-0.05, 1.05)

plt.plot([0, 1], [0, 1], linestyle='--', color='purple', label="Rate of Random Guess")

plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title(f'Final Model ROC Curve (AUC = {roc_auc:.4f})')
plt.legend()
plt.grid()

plt.show()

```



```

[69]: calibration_data = pd.DataFrame({'bin_midpoint': np.arange(0.05, 0.96, 0.1),
                                     'observed_event_percentage': 0.0})
for i in np.arange(0.05, 0.96, 0.1):

```

```

    in_interval = p_1_actual[(p_1_actual['p_1'] >= (i-0.05)) &
↳(p_1_actual['p_1'] <= (i+0.05))].copy()
    in_interval['actual'] = in_interval['actual'].astype(int)
    num_positive_in_interval = len(in_interval.loc[in_interval['actual'] == 1])
    num_in_interval = len(in_interval)

    oep = num_positive_in_interval/num_in_interval

    calibration_data.loc[calibration_data['bin_midpoint'] == i,
↳'observed_event_percentage'] = oep

calibration_data

```

```

[69]:
   bin_midpoint  observed_event_percentage
0          0.05                0.060377
1          0.15                0.252874
2          0.25                0.421751
3          0.35                0.511475
4          0.45                0.583333
5          0.55                0.672897
6          0.65                0.735149
7          0.75                0.807207
8          0.85                0.879227
9          0.95                0.968878

```

```

[89]: roc = sns.lineplot(x='bin_midpoint', y='observed_event_percentage',
↳data=calibration_data)
plt.xlim(-0.05, 1.05)
plt.ylim(-0.05, 1.05)

xlims = roc.get_xlim()
ylims = roc.get_ylim()

min_val = min(xlims[0], ylims[0])
max_val = max(xlims[1], ylims[1])

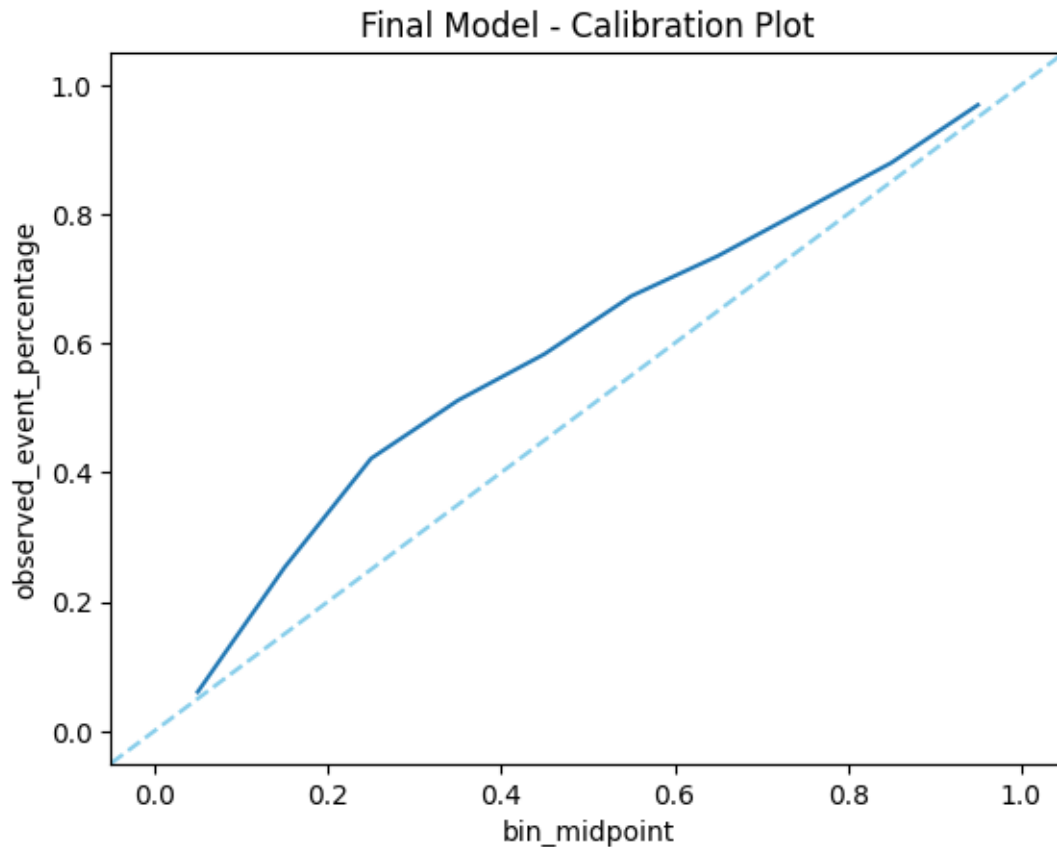
roc.plot([min_val, max_val], [min_val, max_val], linestyle='--',
↳color='skyblue')

roc.set_xlim(xlims)
roc.set_ylim(ylims)

plt.title(f'Final Model - Calibration Plot')

plt.show()

```



```
[87]: seq_model.eval()

X_test_tensor = torch.tensor(X_test_scaled.values, dtype=torch.float32)

integrated_gradients = IntegratedGradients(seq_model)

input_tensor = X_test_tensor[0].unsqueeze(0)

attributions, delta = integrated_gradients.attribute(input_tensor, target=0,
↪return_convergence_delta=True)

attributions = attributions.squeeze().cpu().detach().numpy()

feature_names = X_test_scaled.columns
```

```

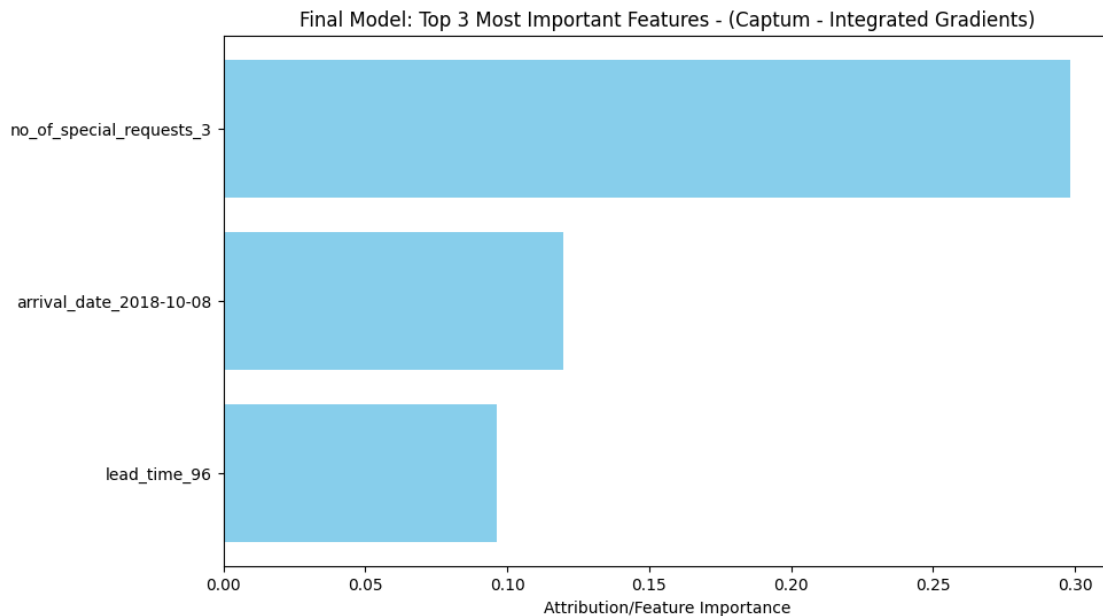
feature_importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": np.abs(attributions)
})

feature_importance_df = feature_importance_df.sort_values(by="Importance",
    ↪ascending=False)

top_3_features = feature_importance_df.head(3)

plt.figure(figsize=(10, 6))
plt.barh(top_3_features['Feature'], top_3_features['Importance'],
    ↪color='skyblue')
plt.xlabel("Attribution/Feature Importance")
plt.title("Final Model: Top 3 Most Important Features - (Captum - Integrated
    ↪Gradients)")
plt.gca().invert_yaxis()
plt.show()

```



```

[ ]: Model 2 using Patience, ReLU and Sigmoid activation function, BCELoss loss
    ↪function,
    and RMSProp as the optimizer with stopping on 5000 epochs

```

```
[25]: seq_model2 = nn.Sequential(
    nn.Linear(4373, 300),
    nn.ReLU(),
    nn.Linear(300, 150),
    nn.ReLU(),
    nn.Linear(150, 1),
    nn.Sigmoid()
)

[26]: def training_loop(n_epochs, optimizer, model, loss_fn, X_tr, X_val, y_tr,
    ↪y_val, patience=15):
    best_val_loss = float("inf")
    counter = 0

    for epoch in range(1, n_epochs + 1):
        seq_model2.train()

        X_tr_p = model(X_tr)
        loss_train = loss_fn(X_tr_p, y_tr.unsqueeze(1))

        optimizer.zero_grad()
        loss_train.backward()
        optimizer.step()

        seq_model2.eval()
        with torch.no_grad():
            X_val_p = model(X_val)
            loss_val = loss_fn(X_val_p, y_val.unsqueeze(1))

        if epoch == 1 or epoch % 100 == 0:
            print(f"Epoch {epoch}, Training loss {loss_train.item():.4f},
    ↪Validation loss {loss_val.item():.4f}")

[27]: optimizer2 = optim.RMSprop(seq_model2.parameters(), lr=1e-3)
training_loop(
    n_epochs = 5000,
    optimizer = optimizer2,
    model = seq_model2,
    loss_fn = nn.BCELoss(),
    X_tr = X_tr,
    X_val = X_val,
    y_tr = y_tr,
    y_val = y_val)
```

```
Epoch 1, Training loss 0.7248, Validation loss 1.0343
Epoch 100, Training loss 0.0141, Validation loss 1.8579
Epoch 200, Training loss 0.0123, Validation loss 2.1920
```

Epoch 300, Training loss 0.0120, Validation loss 2.3963  
Epoch 400, Training loss 0.0116, Validation loss 2.4212  
Epoch 500, Training loss 0.0112, Validation loss 2.5091  
Epoch 600, Training loss 0.0110, Validation loss 2.5242  
Epoch 700, Training loss 0.0109, Validation loss 2.6425  
Epoch 800, Training loss 0.0106, Validation loss 2.6914  
Epoch 900, Training loss 0.0107, Validation loss 2.7362  
Epoch 1000, Training loss 0.0150, Validation loss 3.0125  
Epoch 1100, Training loss 0.0149, Validation loss 3.5907  
Epoch 1200, Training loss 0.0149, Validation loss 3.7614  
Epoch 1300, Training loss 0.0149, Validation loss 3.8648  
Epoch 1400, Training loss 0.0150, Validation loss 3.9081  
Epoch 1500, Training loss 0.0150, Validation loss 3.8977  
Epoch 1600, Training loss 0.0150, Validation loss 3.8718  
Epoch 1700, Training loss 0.0150, Validation loss 3.8455  
Epoch 1800, Training loss 0.0149, Validation loss 3.9038  
Epoch 1900, Training loss 0.0149, Validation loss 3.9573  
Epoch 2000, Training loss 0.0150, Validation loss 4.0051  
Epoch 2100, Training loss 0.0149, Validation loss 4.1574  
Epoch 2200, Training loss 0.0149, Validation loss 4.2190  
Epoch 2300, Training loss 0.0149, Validation loss 4.2482  
Epoch 2400, Training loss 0.0582, Validation loss 4.3123  
Epoch 2500, Training loss 0.0582, Validation loss 4.5628  
Epoch 2600, Training loss 0.0582, Validation loss 4.6746  
Epoch 2700, Training loss 0.0582, Validation loss 4.7798  
Epoch 2800, Training loss 0.0582, Validation loss 4.8037  
Epoch 2900, Training loss 0.0582, Validation loss 4.7936  
Epoch 3000, Training loss 0.0582, Validation loss 4.8135  
Epoch 3100, Training loss 0.0582, Validation loss 4.7860  
Epoch 3200, Training loss 0.0582, Validation loss 4.7424  
Epoch 3300, Training loss 0.0562, Validation loss 4.8130  
Epoch 3400, Training loss 0.0563, Validation loss 4.7118  
Epoch 3500, Training loss 0.0563, Validation loss 4.6221  
Epoch 3600, Training loss 0.0563, Validation loss 4.5459  
Epoch 3700, Training loss 0.0562, Validation loss 4.5523  
Epoch 3800, Training loss 0.0563, Validation loss 4.5615  
Epoch 3900, Training loss 0.0562, Validation loss 4.5760  
Epoch 4000, Training loss 0.1521, Validation loss 4.2645  
Epoch 4100, Training loss 0.1508, Validation loss 4.5627  
Epoch 4200, Training loss 0.1510, Validation loss 4.6928  
Epoch 4300, Training loss 0.1511, Validation loss 4.7600  
Epoch 4400, Training loss 0.1513, Validation loss 4.8511  
Epoch 4500, Training loss 0.1513, Validation loss 4.8895  
Epoch 4600, Training loss 0.1513, Validation loss 4.9682  
Epoch 4700, Training loss 0.1513, Validation loss 4.9852  
Epoch 4800, Training loss 0.1514, Validation loss 4.9875  
Epoch 4900, Training loss 0.1514, Validation loss 5.0036  
Epoch 5000, Training loss 0.1514, Validation loss 4.9695

```
[28]: seq_model2(X_test_tensor).detach().numpy()
```

```
[28]: array([[1.0000000e+00],
          [1.0000000e+00],
          [1.0000000e+00],
          ...,
          [1.0000000e+00],
          [1.5568187e-18],
          [9.999988e-01]], dtype=float32)
```

```
[ ]: Model 2 predictions, ROC AUC, Calibration Plot, and Feature Importance
```

```
[29]: p_1_actual2 = pd.DataFrame({'p_1': seq_model2(X_test_tensor).detach().numpy().
    ↪ reshape(-1), 'actual': y_test_encoded})
p_1_actual2.head()
```

```
[29]:
```

|   | p_1      | actual |
|---|----------|--------|
| 0 | 1.000000 | 1      |
| 1 | 1.000000 | 1      |
| 2 | 1.000000 | 1      |
| 3 | 0.000009 | 1      |
| 4 | 1.000000 | 1      |

```
[30]: roc_list2 = []

actuals_02 = (p_1_actual2['actual'] == 0).sum()
actuals_12 = (p_1_actual2['actual'] == 1).sum()

for i in np.arange(1, -0.01, -0.01):

    over_threshold2 = p_1_actual2[p_1_actual2['p_1'] >= i]

    fp2 = (over_threshold2['actual'] == 0).sum()
    tp2 = (over_threshold2['actual'] == 1).sum()

    fpr2 = fp2 / actuals_02 if actuals_02 > 0 else 0.0
    tpr2 = tp2 / actuals_12 if actuals_12 > 0 else 0.0

    roc_list2.append((i, fpr2, tpr2))

roc_data2 = pd.DataFrame(roc_list2, columns=['threshold', 'fpr', 'tpr'])

roc_data2
```

```
[30]:
```

|   | threshold    | fpr      | tpr      |
|---|--------------|----------|----------|
| 0 | 1.000000e+00 | 0.097986 | 0.594683 |
| 1 | 9.900000e-01 | 0.221919 | 0.885174 |

```

2    9.800000e-01  0.226699  0.893003
3    9.700000e-01  0.230795  0.896428
4    9.600000e-01  0.231137  0.897896
..    ...
96    4.000000e-02  0.316149  0.946828
97    3.000000e-02  0.319222  0.947969
98    2.000000e-02  0.322294  0.949274
99    1.000000e-02  0.326733  0.951068
100 -8.881784e-16  1.000000  1.000000

```

[101 rows x 3 columns]

```
[31]: roc_auc2 = auc(roc_data2['fpr'], roc_data2['tpr'])
print(f"ROC AUC Score: {roc_auc2:.4f}")
```

ROC AUC Score: 0.8457

```
[32]: predicted_labels2 = (p_1_actual2['p_1'] >= 0.5).astype(int)

accuracy2 = (predicted_labels2 == p_1_actual2['actual']).mean()

print(f"Accuracy: {accuracy2:.4f}")
```

Accuracy: 0.8664

```
[33]: y_pred_probs2 = seq_model2(X_test_tensor).detach().numpy()

y_pred2 = (y_pred_probs2 >= 0.5).astype(int)

y_true2 = y_test_tensor.numpy().astype(int)
```

```
[34]: from sklearn.metrics import precision_score, recall_score

precision2 = precision_score(y_true2, y_pred2)
recall2 = recall_score(y_true2, y_pred2)

print(f"Precision: {precision2:.4f}")
print(f"Recall: {recall2:.4f}")
```

Precision: 0.8785

Recall: 0.9315

```
[99]: plt.figure(figsize=(8, 6))
roc2 = sns.lineplot(x='fpr', y='tpr', data=roc_data, marker='o', linestyle='-')

xlims2 = roc.get_xlim()
ylims2 = roc.get_ylim()

plt.xlim(-0.05, 1.05)
```

```

plt.ylim(-0.05, 1.05)

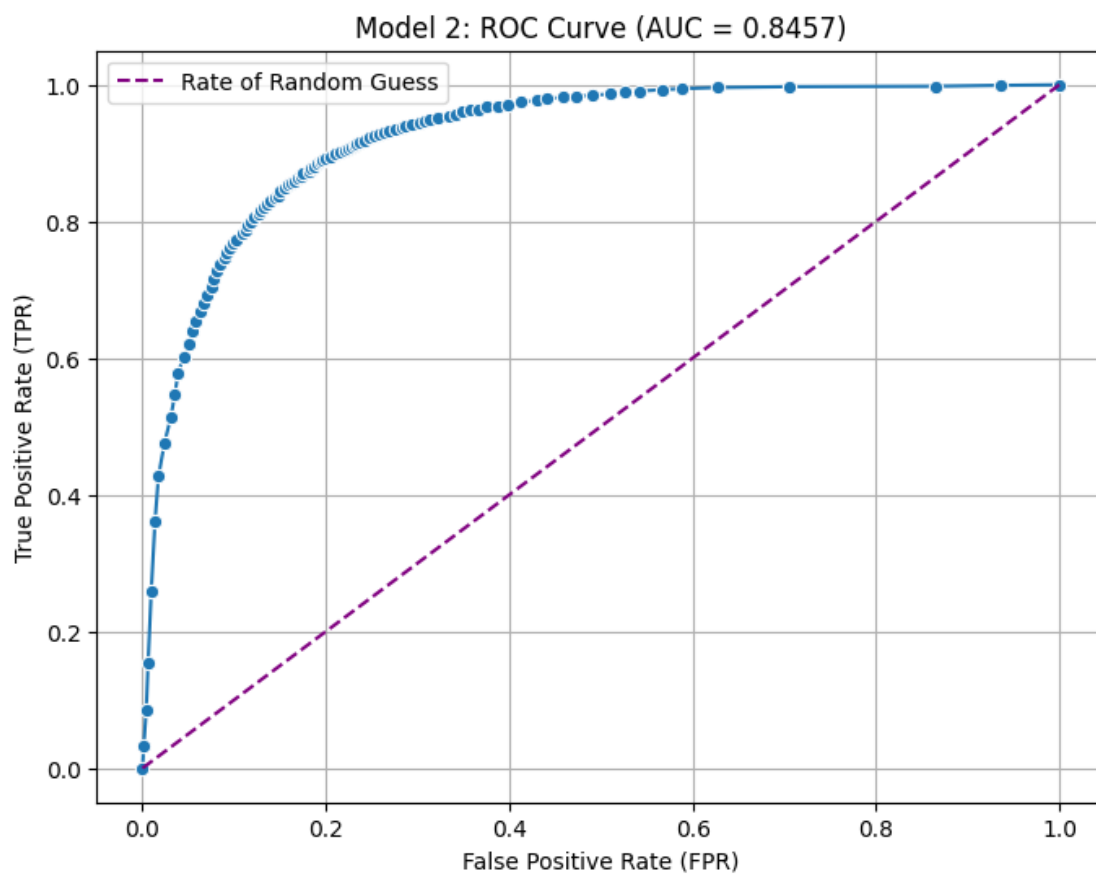
plt.plot([0, 1], [0, 1], linestyle='--', color='purple', label="Rate of Random  

↳ Guess")

plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title(f'Model 2: ROC Curve (AUC = {roc_auc2:.4f})')
plt.legend()
plt.grid()

plt.show()

```



```

[66]: calibration_data2 = pd.DataFrame({'bin_midpoint': np.arange(0.05, 0.96, 0.1),
                                         'observed_event_percentage': 0.0})
for i in np.arange(0.05, 0.96, 0.1):

    in_interval2 = p_1_actual2[(p_1_actual2['p_1'] >= (i-0.05)) &
↳ (p_1_actual2['p_1'] <= (i+0.05))].copy()

```

```

in_interval2['actual'] = in_interval2['actual'].astype(int)
num_positive_in_interval2 = len(in_interval2.loc[in_interval2['actual'] == 1])
num_in_interval2 = len(in_interval2)

oep2 = num_positive_in_interval2/num_in_interval2

calibration_data2.loc[calibration_data2['bin_midpoint'] == i,
'observed_event_percentage'] = oep2

calibration_data2

```

```

[66]:
   bin_midpoint  observed_event_percentage
0          0.05                0.144477
1          0.15                0.524590
2          0.25                0.333333
3          0.35                0.400000
4          0.45                0.393939
5          0.55                0.642857
6          0.65                0.586207
7          0.75                0.483871
8          0.85                0.693182
9          0.95                0.886753

```

```

[88]: roc2 = sns.lineplot(x='bin_midpoint', y='observed_event_percentage',
    data=calibration_data2)
plt.xlim(-0.05, 1.05)
plt.ylim(-0.05, 1.05)

xlims2 = roc2.get_xlim()
ylims2 = roc2.get_ylim()

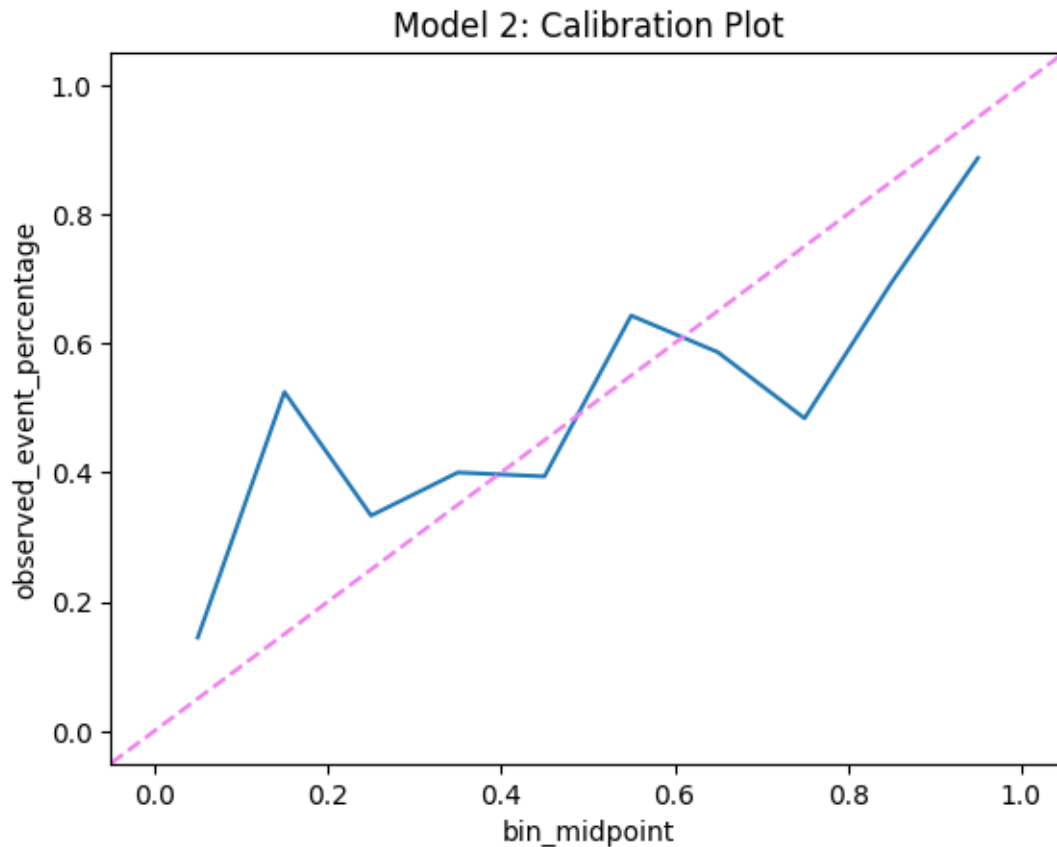
min_val2 = min(xlims2[0], ylims2[0])
max_val2 = max(xlims2[1], ylims2[1])

roc2.plot([min_val2, max_val2], [min_val2, max_val2], linestyle='--',
    color='violet')

roc2.set_xlim(xlims2)
roc2.set_ylim(ylims2)

plt.title(f'Model 2 - Calibration Plot')
plt.show()

```



```
[90]: seq_model2.eval()

X_test_tensor2 = torch.tensor(X_test_scaled.values, dtype=torch.float32)

integrated_gradients2 = IntegratedGradients(seq_model2)

input_tensor2 = X_test_tensor2[0].unsqueeze(0)

attributions2, delta2 = integrated_gradients2.attribute(input_tensor2,
    ↪target=0, return_convergence_delta=True)

attributions2 = attributions2.squeeze().cpu().detach().numpy()

feature_names2 = X_test_scaled.columns
feature_importance_df2 = pd.DataFrame({
    "Feature": feature_names2,
    "Importance": np.abs(attributions2)
```

```

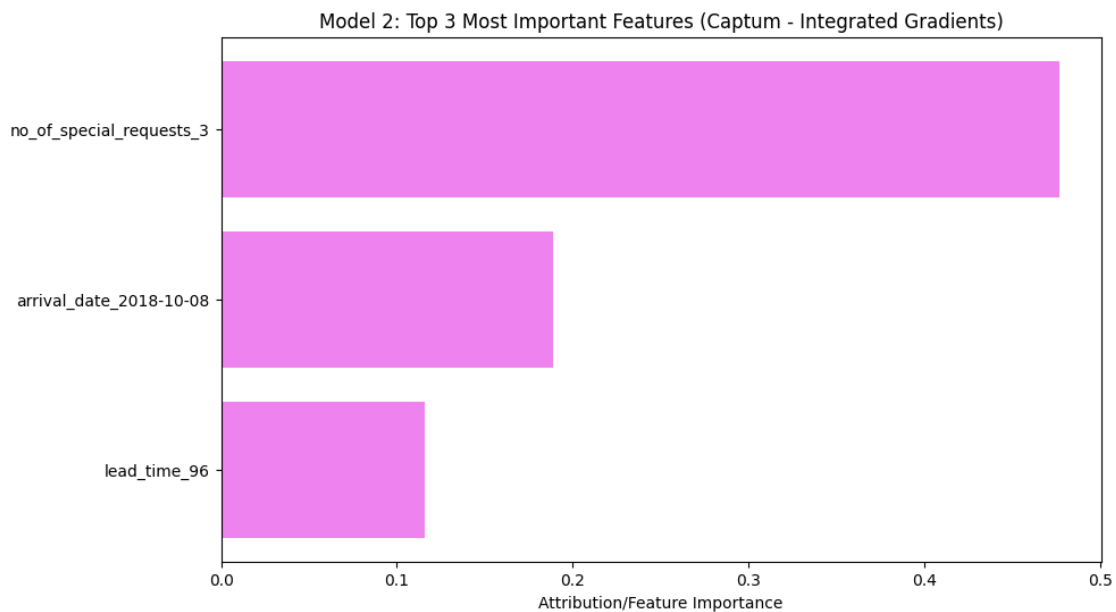
})

feature_importance_df2 = feature_importance_df2.sort_values(by="Importance",
↳ascending=False)

top_3_features2 = feature_importance_df2.head(3)

plt.figure(figsize=(10, 6))
plt.barh(top_3_features2['Feature'], top_3_features2['Importance'],
↳color='violet')
plt.xlabel("Attribution/Feature Importance")
plt.title("Model 2: Top 3 Most Important Features (Captum - Integrated
↳Gradients)")
plt.gca().invert_yaxis() # Highest importance at top
plt.show()

```



```

[ ]: Model 3 using Patience, Tanh and Sigmoid activation function, MSELoss loss
↳function,
and RMSProp as the optimizer with 5000 epochs

```

```

[37]: seq_model3 = nn.Sequential(
    nn.Linear(4373, 300),
    nn.Tanh(),
    nn.Linear(300, 150),

```

```

nn.Tanh(),
nn.Linear(150, 1),
nn.Sigmoid()
)

```

```

[38]: def training_loop(n_epochs, optimizer, model, loss_fn, X_tr, X_val, y_tr,
    ↪ y_val, patience=15):
    best_val_loss = float("inf")
    counter = 0

    for epoch in range(1, n_epochs + 1):
        seq_model3.train()

        X_tr_p = model(X_tr)
        loss_train = loss_fn(X_tr_p, y_tr.unsqueeze(1))

        optimizer.zero_grad()
        loss_train.backward()
        optimizer.step()

        seq_model3.eval()
        with torch.no_grad():
            X_val_p = model(X_val)
            loss_val = loss_fn(X_val_p, y_val.unsqueeze(1))

        if epoch == 1 or epoch % 100 == 0:
            print(f"Epoch {epoch}, Training loss {loss_train.item():.4f},
    ↪ Validation loss {loss_val.item():.4f}")

```

```

[39]: optimizer3 = optim.RMSprop(seq_model3.parameters(), lr=1e-3)
training_loop(
    n_epochs = 5000,
    optimizer = optimizer3,
    model = seq_model3,
    loss_fn = nn.MSELoss(),
    X_tr = X_tr,
    X_val = X_val,
    y_tr = y_tr,
    y_val = y_val)

```

```

Epoch 1, Training loss 0.2498, Validation loss 0.1973
Epoch 100, Training loss 0.0181, Validation loss 0.1661
Epoch 200, Training loss 0.0155, Validation loss 0.1704
Epoch 300, Training loss 0.0149, Validation loss 0.1693
Epoch 400, Training loss 0.0142, Validation loss 0.1703
Epoch 500, Training loss 0.0136, Validation loss 0.1695
Epoch 600, Training loss 0.0134, Validation loss 0.1698

```

Epoch 700, Training loss 0.0132, Validation loss 0.1707  
Epoch 800, Training loss 0.0123, Validation loss 0.1696  
Epoch 900, Training loss 0.0118, Validation loss 0.1703  
Epoch 1000, Training loss 0.0116, Validation loss 0.1699  
Epoch 1100, Training loss 0.0112, Validation loss 0.1695  
Epoch 1200, Training loss 0.0108, Validation loss 0.1699  
Epoch 1300, Training loss 0.0103, Validation loss 0.1687  
Epoch 1400, Training loss 0.0098, Validation loss 0.1702  
Epoch 1500, Training loss 0.0094, Validation loss 0.1703  
Epoch 1600, Training loss 0.0090, Validation loss 0.1708  
Epoch 1700, Training loss 0.0088, Validation loss 0.1711  
Epoch 1800, Training loss 0.0085, Validation loss 0.1692  
Epoch 1900, Training loss 0.0084, Validation loss 0.1693  
Epoch 2000, Training loss 0.0081, Validation loss 0.1686  
Epoch 2100, Training loss 0.0079, Validation loss 0.1690  
Epoch 2200, Training loss 0.0074, Validation loss 0.1685  
Epoch 2300, Training loss 0.0071, Validation loss 0.1707  
Epoch 2400, Training loss 0.0068, Validation loss 0.1712  
Epoch 2500, Training loss 0.0068, Validation loss 0.1713  
Epoch 2600, Training loss 0.0064, Validation loss 0.1725  
Epoch 2700, Training loss 0.0062, Validation loss 0.1720  
Epoch 2800, Training loss 0.0062, Validation loss 0.1737  
Epoch 2900, Training loss 0.0055, Validation loss 0.1733  
Epoch 3000, Training loss 0.0054, Validation loss 0.1735  
Epoch 3100, Training loss 0.0052, Validation loss 0.1734  
Epoch 3200, Training loss 0.0051, Validation loss 0.1735  
Epoch 3300, Training loss 0.0048, Validation loss 0.1743  
Epoch 3400, Training loss 0.0047, Validation loss 0.1735  
Epoch 3500, Training loss 0.0046, Validation loss 0.1738  
Epoch 3600, Training loss 0.0045, Validation loss 0.1737  
Epoch 3700, Training loss 0.0045, Validation loss 0.1737  
Epoch 3800, Training loss 0.0045, Validation loss 0.1735  
Epoch 3900, Training loss 0.0045, Validation loss 0.1741  
Epoch 4000, Training loss 0.0044, Validation loss 0.1735  
Epoch 4100, Training loss 0.0043, Validation loss 0.1738  
Epoch 4200, Training loss 0.0043, Validation loss 0.1741  
Epoch 4300, Training loss 0.0043, Validation loss 0.1741  
Epoch 4400, Training loss 0.0043, Validation loss 0.1739  
Epoch 4500, Training loss 0.0042, Validation loss 0.1732  
Epoch 4600, Training loss 0.0039, Validation loss 0.1731  
Epoch 4700, Training loss 0.0038, Validation loss 0.1725  
Epoch 4800, Training loss 0.0038, Validation loss 0.1735  
Epoch 4900, Training loss 0.0037, Validation loss 0.1733  
Epoch 5000, Training loss 0.0037, Validation loss 0.1760

```
[40]: seq_model3(X_test_tensor).detach().numpy()
```

```
[40]: array([[9.9987459e-01],
          [9.9986017e-01],
          [9.9953663e-01],
          ...,
          [9.9979538e-01],
          [9.9613600e-05],
          [9.9987555e-01]], dtype=float32)
```

```
[ ]: Model 3 predictions, ROC AUC, Calibration Plot, and Feature Importance
```

```
[41]: p_1_actual3 = pd.DataFrame({'p_1': seq_model3(X_test_tensor).detach().numpy().
    ↪reshape(-1), 'actual': y_test_encoded})
p_1_actual3.head()
```

```
[41]:
```

|   | p_1      | actual |
|---|----------|--------|
| 0 | 0.999875 | 1      |
| 1 | 0.999860 | 1      |
| 2 | 0.999537 | 1      |
| 3 | 0.004278 | 1      |
| 4 | 0.999627 | 1      |

```
[42]: roc_list3 = []

actuals_03 = (p_1_actual3['actual'] == 0).sum()
actuals_13 = (p_1_actual3['actual'] == 1).sum()

for i in np.arange(1, -0.01, -0.01):

    over_threshold3 = p_1_actual3[p_1_actual3['p_1'] >= i]

    fp3 = (over_threshold3['actual'] == 0).sum()
    tp3 = (over_threshold3['actual'] == 1).sum()

    fpr3 = fp3 / actuals_03 if actuals_03 > 0 else 0.0
    tpr3 = tp3 / actuals_13 if actuals_13 > 0 else 0.0

    roc_list3.append((i, fpr3, tpr3))

roc_data3 = pd.DataFrame(roc_list3, columns=['threshold', 'fpr', 'tpr'])

roc_data3
```

```
[42]:
```

|   | threshold    | fpr      | tpr      |
|---|--------------|----------|----------|
| 0 | 1.000000e+00 | 0.000000 | 0.000000 |
| 1 | 9.900000e-01 | 0.125982 | 0.766596 |
| 2 | 9.800000e-01 | 0.134517 | 0.782417 |
| 3 | 9.700000e-01 | 0.140321 | 0.792367 |

```

4      9.600000e-01  0.143394  0.797423
..      ...      ...      ...
96      4.000000e-02  0.272789  0.906214
97      3.000000e-02  0.278935  0.909476
98      2.000000e-02  0.290201  0.913554
99      1.000000e-02  0.311369  0.921709
100     -8.881784e-16  1.000000  1.000000

```

[101 rows x 3 columns]

```
[43]: roc_auc3 = auc(roc_data3['fpr'], roc_data3['tpr'])
      print(f"ROC AUC Score: {roc_auc3:.4f}")
```

ROC AUC Score: 0.8702

```
[44]: predicted_labels3 = (p_1_actual3['p_1'] >= 0.5).astype(int)

      accuracy3 = (predicted_labels3 == p_1_actual3['actual']).mean()

      print(f"Accuracy: {accuracy3:.4f}")
```

Accuracy: 0.8411

```
[45]: y_pred_probs3 = seq_model3(X_test_tensor).detach().numpy()

      y_pred3 = (y_pred_probs3 >= 0.5).astype(int)

      y_true3 = y_test_tensor.numpy().astype(int)
```

```
[46]: from sklearn.metrics import precision_score, recall_score

      precision3 = precision_score(y_true3, y_pred3)
      recall3 = recall_score(y_true3, y_pred3)

      print(f"Precision: {precision3:.4f}")
      print(f"Recall: {recall3:.4f}")
```

Precision: 0.8969

Recall: 0.8645

```
[91]: plt.figure(figsize=(8, 6))
      roc3 = sns.lineplot(x='fpr', y='tpr', data=roc_data, marker='o', linestyle='-')

      xlims3 = roc.get_xlim()
      ylims3 = roc.get_ylim()

      plt.xlim(-0.05, 1.05)
      plt.ylim(-0.05, 1.05)
```

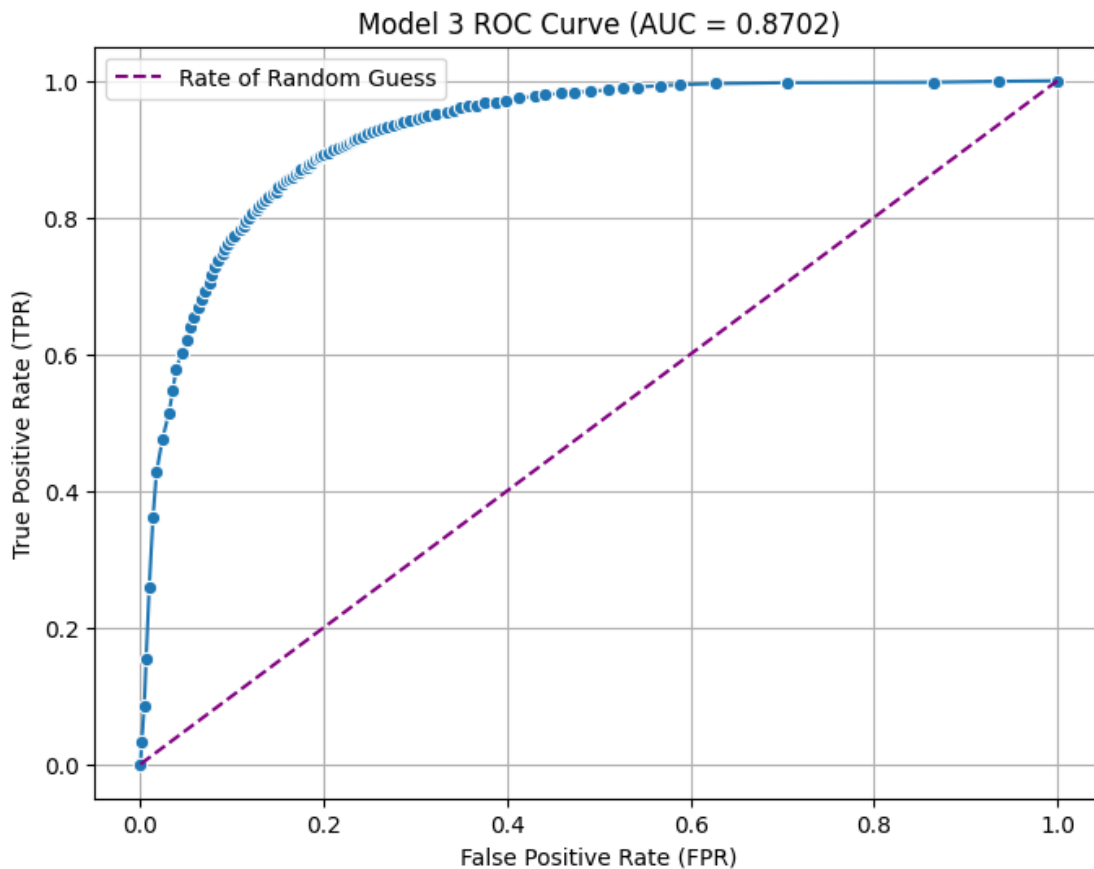
```

plt.plot([0, 1], [0, 1], linestyle='--', color='purple', label="Rate of Random_
↳Guess")

plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title(f'Model 3 ROC Curve (AUC = {roc_auc3:.4f})')
plt.legend()
plt.grid()

plt.show()

```



```

[48]: calibration_data3 = pd.DataFrame({'bin_midpoint': np.arange(0.05, 0.96, 0.1),
                                         'observed_event_percentage': 0.0})
for i in np.arange(0.05, 0.96, 0.1):

    in_interval3 = p_1_actual3[(p_1_actual3['p_1'] >= (i-0.05)) &
↳(p_1_actual3['p_1'] <= (i+0.05))].copy()
    in_interval3['actual'] = in_interval3['actual'].astype(int)

```

```

    num_positive_in_interval3 = len(in_interval3.loc[in_interval3['actual'] == 1])
    num_in_interval3 = len(in_interval3)

    oep3 = num_positive_in_interval3/num_in_interval3

    calibration_data3.loc[calibration_data3['bin_midpoint'] == i,
    'observed_event_percentage'] = oep3

calibration_data3

```

```

[48]:   bin_midpoint  observed_event_percentage
0         0.05                0.229005
1         0.15                0.592233
2         0.25                0.600000
3         0.35                0.539474
4         0.45                0.555556
5         0.55                0.581081
6         0.65                0.656566
7         0.75                0.670732
8         0.85                0.701389
9         0.95                0.913975

```

```

[92]: roc3 = sns.lineplot(x='bin_midpoint', y='observed_event_percentage',
    data=calibration_data3)
plt.xlim(-0.05, 1.05)
plt.ylim(-0.05, 1.05)

xlims3 = roc3.get_xlim()
ylims3 = roc3.get_ylim()

min_val3 = min(xlims3[0], ylims3[0])
max_val3 = max(xlims3[1], ylims3[1])

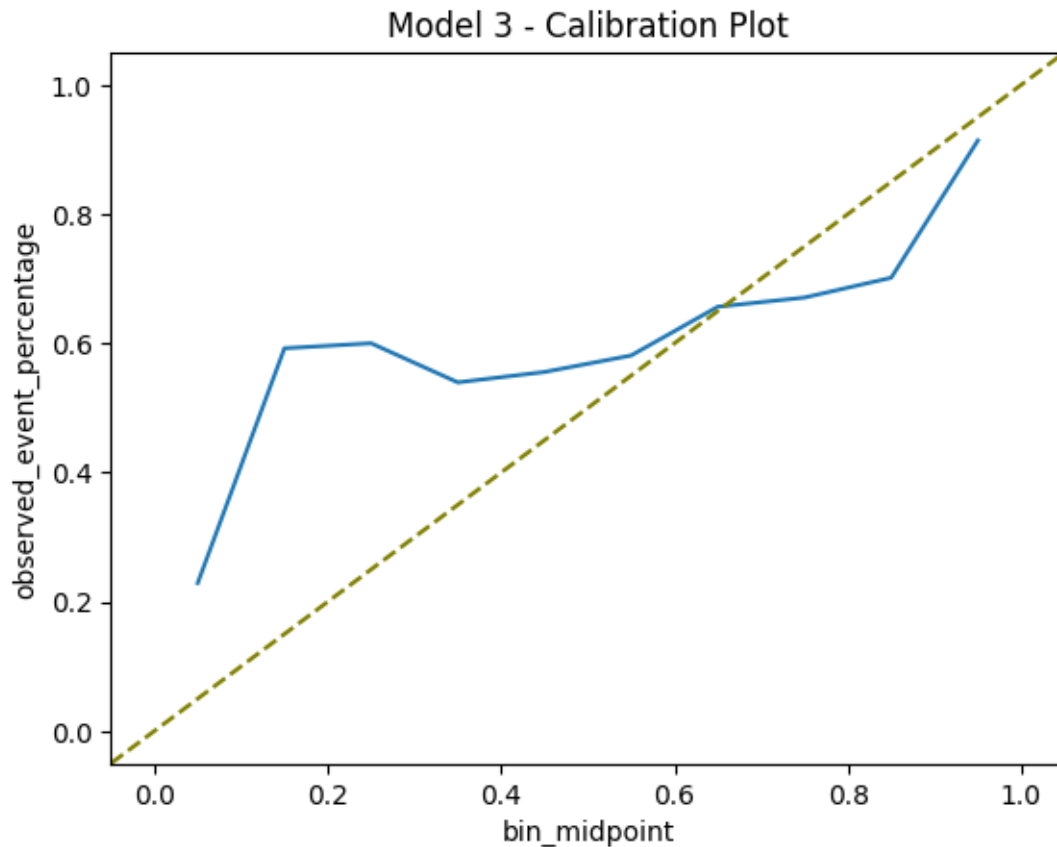
roc3.plot([min_val3, max_val3], [min_val3, max_val3], linestyle='--',
    color='olive')

roc3.set_xlim(xlims3)
roc3.set_ylim(ylims3)

plt.title(f'Model 3 - Calibration Plot')

plt.show()

```



```
[96]: seq_model3.eval()

X_test_tensor3 = torch.tensor(X_test_scaled.values, dtype=torch.float32)

integrated_gradients3 = IntegratedGradients(seq_model3)

input_tensor3 = X_test_tensor3[0].unsqueeze(0)

attributions3, delta3 = integrated_gradients3.attribute(input_tensor3,
    ↪target=0, return_convergence_delta=True)

attributions3 = attributions3.squeeze().cpu().detach().numpy()

feature_names3 = X_test_scaled.columns
feature_importance_df3 = pd.DataFrame({
    "Feature": feature_names3,
    "Importance": np.abs(attributions3)
})
```

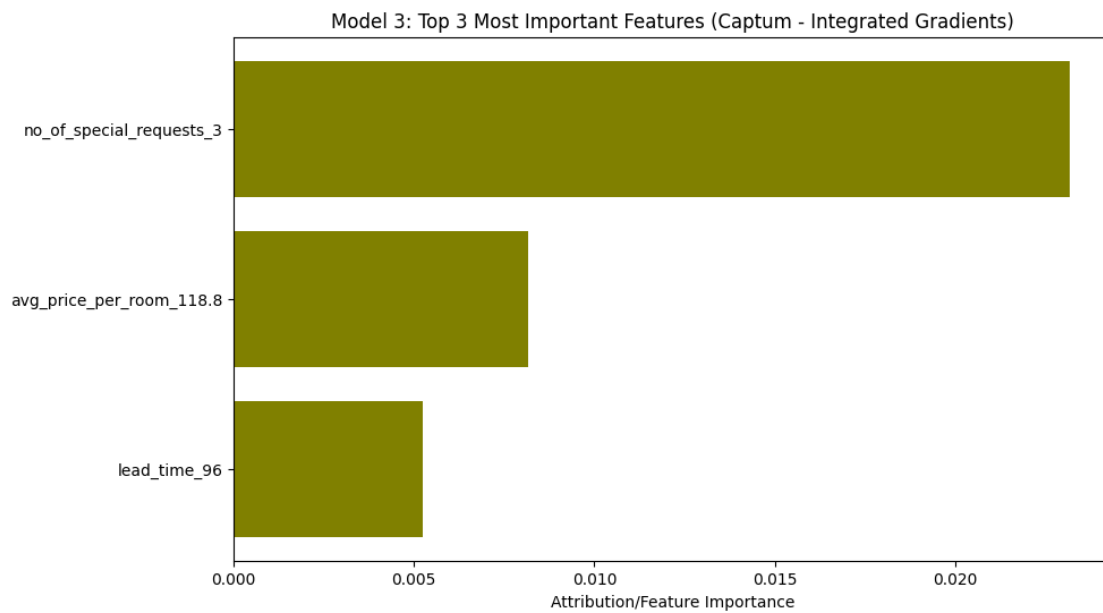
```

feature_importance_df3 = feature_importance_df3.sort_values(by="Importance",
↳ascending=False)

top_3_features3 = feature_importance_df3.head(3)

plt.figure(figsize=(10, 6))
plt.barh(top_3_features3['Feature'], top_3_features3['Importance'],
↳color='olive')
plt.xlabel("Attribution/Feature Importance")
plt.title("Model 3: Top 3 Most Important Features (Captum - Integrated
↳Gradients)")
plt.gca().invert_yaxis()
plt.show()

```



```

[ ]: Model 4 using Patience, Tanh and Sigmoid activation function, MSELoss loss
↳function,
and Adam as the optimizer with stopping on 200 epochs

```

```

[97]: seq_model4 = nn.Sequential(
    nn.Linear(4373, 100),
    nn.Tanh(),
    nn.Dropout(0.5),
    nn.Linear(100, 50),
    nn.Tanh(),
    nn.Dropout(0.5),
    nn.Linear(50, 1),

```

```

nn.Sigmoid()
)

```

```

[52]: def training_loop(n_epochs, optimizer, model, loss_fn, X_tr, X_val, y_tr,
    ↪y_val, patience=100):
    best_val_loss = float("inf")
    counter = 0

    for epoch in range(1, n_epochs + 1):
        seq_model4.train()

        X_tr_p = model(X_tr)
        loss_train = loss_fn(X_tr_p, y_tr.unsqueeze(1))

        optimizer.zero_grad()
        loss_train.backward()
        optimizer.step()

        seq_model4.eval()
        with torch.no_grad():
            X_val_p = model(X_val)
            loss_val = loss_fn(X_val_p, y_val.unsqueeze(1))

        if epoch == 1 or epoch % 10 == 0:
            print(f"Epoch {epoch}, Training loss {loss_train.item():.4f},
    ↪Validation loss {loss_val.item():.4f}")

        if loss_val.item() < best_val_loss:
            best_val_loss = loss_val.item()
            counter = 0
        else:
            counter += 1

        if counter >= patience:
            print(f"Overfitting detected at epoch {epoch}. Training concluded.")
            print(f"Final Epoch: {epoch}, Training Loss: {loss_train.item():.
    ↪4f}, Validation Loss: {loss_val.item():.4f}")
            break

```

```

[53]: optimizer4 = optimizer = optim.Adam(seq_model4.parameters(), lr=1e-3,
    ↪weight_decay=1e-4)
training_loop(
    n_epochs = 200,
    optimizer = optimizer4,
    model = seq_model4,
    loss_fn = nn.MSELoss(),

```

```
X_tr = X_tr,
X_val = X_val,
y_tr = y_tr,
y_val = y_val)
```

```
Epoch 1, Training loss 0.2664, Validation loss 0.2533
Epoch 10, Training loss 0.1567, Validation loss 0.1975
Epoch 20, Training loss 0.1060, Validation loss 0.1830
Epoch 30, Training loss 0.0857, Validation loss 0.1764
Epoch 40, Training loss 0.0746, Validation loss 0.1705
Epoch 50, Training loss 0.0680, Validation loss 0.1660
Epoch 60, Training loss 0.0646, Validation loss 0.1636
Epoch 70, Training loss 0.0613, Validation loss 0.1625
Epoch 80, Training loss 0.0586, Validation loss 0.1620
Epoch 90, Training loss 0.0562, Validation loss 0.1619
Epoch 100, Training loss 0.0539, Validation loss 0.1613
Epoch 110, Training loss 0.0529, Validation loss 0.1615
Epoch 120, Training loss 0.0505, Validation loss 0.1608
Epoch 130, Training loss 0.0490, Validation loss 0.1608
Epoch 140, Training loss 0.0482, Validation loss 0.1612
Epoch 150, Training loss 0.0455, Validation loss 0.1617
Epoch 160, Training loss 0.0440, Validation loss 0.1612
Epoch 170, Training loss 0.0426, Validation loss 0.1613
Epoch 180, Training loss 0.0409, Validation loss 0.1615
Epoch 190, Training loss 0.0400, Validation loss 0.1628
Epoch 200, Training loss 0.0385, Validation loss 0.1623
```

```
[54]: seq_model4(X_test_tensor).detach().numpy()
```

```
[54]: array([[0.99763656],
            [0.99344313],
            [0.9439868 ],
            ...,
            [0.9986205 ],
            [0.00111948],
            [0.9438613 ]], dtype=float32)
```

```
[ ]: Model 4 predictions, ROC AUC, Calibration Plot, and Feature Importance
```

```
[55]: p_1_actual4 = pd.DataFrame({'p_1': seq_model4(X_test_tensor).detach().numpy().
    ↪reshape(-1), 'actual': y_test_encoded})
p_1_actual4.head()
```

```
[55]:
```

|   | p_1      | actual |
|---|----------|--------|
| 0 | 0.997637 | 1      |
| 1 | 0.993443 | 1      |
| 2 | 0.943987 | 1      |
| 3 | 0.388076 | 1      |

4 0.995000 1

```
[56]: roc_list4 = []

actuals_04 = (p_1_actual4['actual'] == 0).sum()
actuals_14 = (p_1_actual4['actual'] == 1).sum()

for i in np.arange(1, -0.01, -0.01):

    over_threshold4 = p_1_actual4[p_1_actual4['p_1'] >= i]

    fp4 = (over_threshold4['actual'] == 0).sum()
    tp4 = (over_threshold4['actual'] == 1).sum()

    fpr4 = fp4 / actuals_04 if actuals_04 > 0 else 0.0
    tpr4 = tp4 / actuals_14 if actuals_14 > 0 else 0.0

    roc_list4.append((i, fpr4, tpr4))

roc_data4 = pd.DataFrame(roc_list4, columns=['threshold', 'fpr', 'tpr'])

roc_data4
```

```
[56]:
```

|     | threshold     | fpr      | tpr      |
|-----|---------------|----------|----------|
| 0   | 1.000000e+00  | 0.000000 | 0.000000 |
| 1   | 9.900000e-01  | 0.064527 | 0.573153 |
| 2   | 9.800000e-01  | 0.081256 | 0.657478 |
| 3   | 9.700000e-01  | 0.092182 | 0.694014 |
| 4   | 9.600000e-01  | 0.101741 | 0.718969 |
| ..  | ...           | ...      | ...      |
| 96  | 4.000000e-02  | 0.395698 | 0.934758 |
| 97  | 3.000000e-02  | 0.430181 | 0.939488 |
| 98  | 2.000000e-02  | 0.482076 | 0.945686 |
| 99  | 1.000000e-02  | 0.572209 | 0.954983 |
| 100 | -8.881784e-16 | 1.000000 | 1.000000 |

[101 rows x 3 columns]

```
[57]: roc_auc4 = auc(roc_data4['fpr'], roc_data4['tpr'])
print(f"ROC AUC Score: {roc_auc4:.4f}")
```

ROC AUC Score: 0.8864

```
[58]: predicted_labels4 = (p_1_actual4['p_1'] >= 0.5).astype(int)

accuracy4 = (predicted_labels4 == p_1_actual4['actual']).mean()

print(f"Accuracy: {accuracy4:.4f}")
```

Accuracy: 0.8478

```
[59]: y_pred_probs4 = seq_model4(X_test_tensor).detach().numpy()

y_pred4 = (y_pred_probs4 >= 0.5).astype(int)

y_true4 = y_test_tensor.numpy().astype(int)
```

```
[60]: precision4 = precision_score(y_true4, y_pred4)
recall4 = recall_score(y_true4, y_pred4)

print(f"Precision: {precision4:.4f}")
print(f"Recall: {recall4:.4f}")
```

Precision: 0.9023

Recall: 0.8692

```
[93]: plt.figure(figsize=(8, 6))
roc4 = sns.lineplot(x='fpr', y='tpr', data=roc_data4, marker='o', linestyle='-')

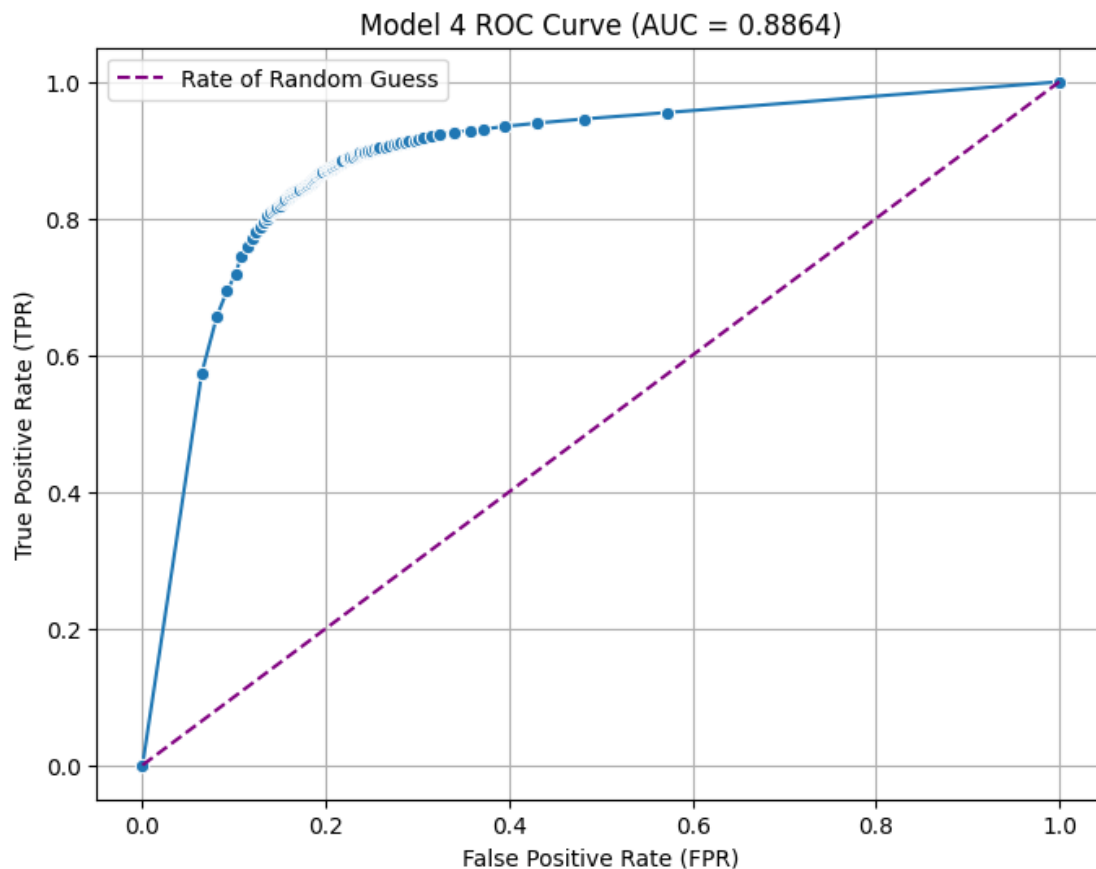
xlims4 = roc.get_xlim()
ylims4 = roc.get_ylim()

plt.xlim(-0.05, 1.05)
plt.ylim(-0.05, 1.05)

plt.plot([0, 1], [0, 1], linestyle='--', color='purple', label="Rate of Random
↳ Guess")

plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title(f'Model 4 ROC Curve (AUC = {roc_auc4:.4f})')
plt.legend()
plt.grid()

plt.show()
```



```
[63]: calibration_data4 = pd.DataFrame({'bin_midpoint': np.arange(0.05, 0.96, 0.1),
                                       'observed_event_percentage': 0.0})
for i in np.arange(0.05, 0.96, 0.1):

    in_interval4 = p_1_actual4[(p_1_actual4['p_1'] >= (i-0.05)) &
    ↪(p_1_actual4['p_1'] <= (i+0.05))].copy()
    in_interval4['actual'] = in_interval4['actual'].astype(int)
    num_positive_in_interval4 = len(in_interval4.loc[in_interval4['actual'] ==
    ↪1])
    num_in_interval4 = len(in_interval4)

    oep4 = num_positive_in_interval4/num_in_interval4

    calibration_data4.loc[calibration_data4['bin_midpoint'] == i,
    ↪'observed_event_percentage'] = oep4

calibration_data4
```

```
[63]:
```

|   | bin_midpoint | observed_event_percentage |
|---|--------------|---------------------------|
| 0 | 0.05         | 0.198266                  |
| 1 | 0.15         | 0.391111                  |
| 2 | 0.25         | 0.467153                  |
| 3 | 0.35         | 0.548872                  |
| 4 | 0.45         | 0.606557                  |
| 5 | 0.55         | 0.693069                  |
| 6 | 0.65         | 0.700000                  |
| 7 | 0.75         | 0.652174                  |
| 8 | 0.85         | 0.749104                  |
| 9 | 0.95         | 0.926356                  |

```
[95]: roc4 = sns.lineplot(x='bin_midpoint', y='observed_event_percentage',
    ↪data=calibration_data4)
plt.xlim(-0.05, 1.05)
plt.ylim(-0.05, 1.05)

xlims4 = roc4.get_xlim()
ylims4 = roc4.get_ylim()

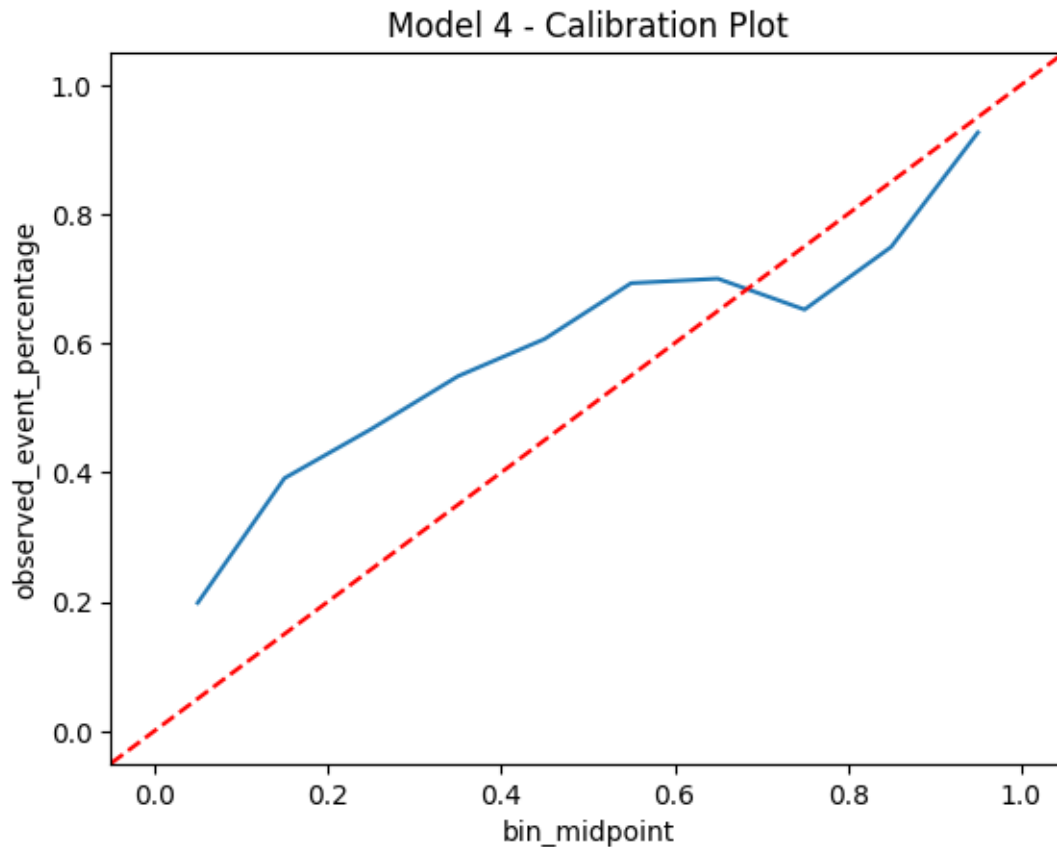
min_val4 = min(xlims4[0], ylims4[0])
max_val4 = max(xlims4[1], ylims4[1])

roc4.plot([min_val4, max_val4], [min_val4, max_val4], linestyle='--',
    ↪color='red')

roc4.set_xlim(xlims4)
roc4.set_ylim(ylims4)

plt.title(f'Model 4 - Calibration Plot')

plt.show()
```



```
[98]: seq_model4.eval()

X_test_tensor4 = torch.tensor(X_test_scaled.values, dtype=torch.float32)

integrated_gradients4 = IntegratedGradients(seq_model4)

input_tensor4 = X_test_tensor4[0].unsqueeze(0)

attributions4, delta4 = integrated_gradients4.attribute(input_tensor4,
    ↪target=0, return_convergence_delta=True)

attributions4 = attributions4.squeeze().cpu().detach().numpy()

feature_names4 = X_test_scaled.columns
feature_importance_df4 = pd.DataFrame({
    "Feature": feature_names4,
    "Importance": np.abs(attributions4)
})
```

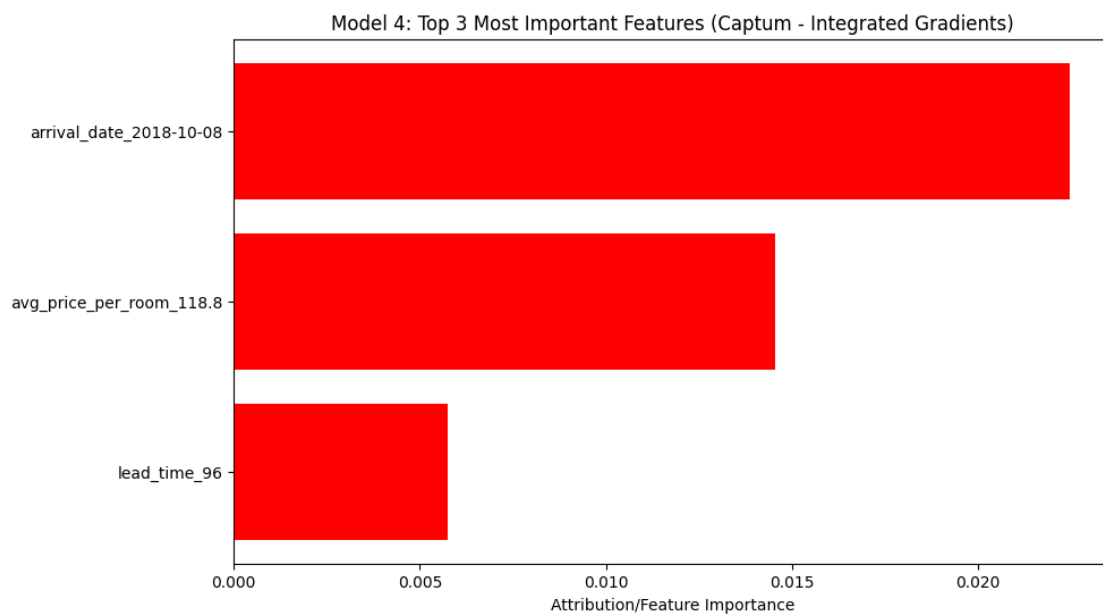
```

feature_importance_df4 = feature_importance_df4.sort_values(by="Importance",
    ↪ascending=False)

top_3_features4 = feature_importance_df4.head(3)

plt.figure(figsize=(10, 6))
plt.barh(top_3_features4['Feature'], top_3_features4['Importance'], color='red')
plt.xlabel("Attribution/Feature Importance")
plt.title("Model 4: Top 3 Most Important Features (Captum - Integrated_
    ↪Gradients)")
plt.gca().invert_yaxis()
plt.show()

```



[ ]: